SENTINELX: CYBER ATTACK INTELLIGENCE AND THREAT HUNTING PLATFORM

PROJECT REPORT (REVIEW 2)

Submitted by

Shlok Gupta (24BCI0061)

Shreya Bhushan (24BCI0086)

Yashneil (24BDS0158)

Prepared For

DATABASE SYSTEMS (BCSE302P) - Lab PROJECT

Submitted To

Dr. Anand Bihari

School of Computer Science and Engineering

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Abstract

Most organisations already collect a lot of security data, such as firewall logs, intrusion alerts and lists of known bad IP addresses from threat feeds. The problem is that this data sits in different places, and news about a bad IP usually arrives days after that IP has already been probing the network. SentinelX is a small Security Operations Centre (SOC) built around one relational database, so that all of this is stored once, linked properly and searched with plain SQL.

The database has 16 MySQL tables in Third Normal Form (3NF). Review 1 covered the ER diagram, the tables and their constraints. This Review 2 report shows the design working: every statement in it was run on MySQL 8.0.46, and each screenshot is the real output of that run. We loaded a test dataset of 50,021 network log lines, 30 attacks, 12 indicators of compromise and 7 incidents, and ran 40 queries across the seven required categories, 11 multi-table business queries, 5 views, 2 functions, 4 stored procedures (2 of them with cursors), 4 triggers, transactions with savepoints, an index comparison with EXPLAIN, and a role-based security setup with three roles and three users. Two features are our own: a retro-hunt trigger that searches old logs as soon as a new bad IP is added, and a "time machine" that tells apart what we knew at the time of an attack from what we only learned later.

Table of Contents

1. Introduction 4
2. Objectives 4
3. Project Scope 4
4. Key Contacts and Stakeholders 4
5. Project Resource Requirements 5
6. System Architecture 5
7. Module-wise Functional Description 5
8. ER Diagram 7
9. Tables and Constraints 10
10. Data Population (INSERT, UPDATE, DELETE) 20
11. SQL Implementation 23
12. Advanced SQL Implementation 40
13. Transactions 68
14. Indexes and EXPLAIN 69
15. Design Changes since Review 1 73
16. Review 2 Requirements Checklist 73
17. Limitations and Future Scope 74
18. Conclusion 74
19. Work Break Down 74
20. Review Evaluation 75

1. Introduction

Anyone who has seen how a small security team works knows the routine. An alert fires, someone opens the firewall console, then the IDS, then a spreadsheet of known bad IPs, and tries to match timestamps by hand. Each tool keeps its own copy of the facts, so the same attacker or server shows up in several places with slightly different details. Threat feeds also tend to be late. By the time an IP address appears on a blocklist it has often been scanning people for weeks, and nobody goes back to check last month's logs.

We picked this topic for our DBMS lab project because it fits a relational database very naturally. There are clear entities (servers, attackers, malware, vulnerabilities, incidents), clear relationships between them (an attack hits one server, an incident can group many attacks) and good reasons to use constraints, triggers, transactions and indexes. Checking old logs for a newly reported bad IP is still a manual job in most tools, so we made the database do that part by itself, and that became the main idea of the project.

2. Objectives

- Design a normalised relational schema in Third Normal Form (3NF) for a security operations centre that avoids update, insert and delete problems.
- Draw a clean ER diagram that keeps only the attributes we really use, and make sure it matches the tables we actually built.
- Keep every column only as large as it needs to be and back every rule with a constraint.
- Let the database do routine work by itself, like opening incidents for serious attacks and checking old logs when a new bad IP is added.
- Prove the design with real execution: queries, views, routines, triggers, transactions and security, each with a screenshot of its output (Review 2).

3. Project Scope

- Basic data about servers and devices (assets), CVEs, attack types mapped to MITRE ATT&CK, attacker groups and malware.
- Storing raw network logs and detected attacks, with indexes on time and source IP so searches over the large tables stay fast.
- Dashboard and search views over attacks, assets and risk.
- Incident handling: automatic creation, assigning an analyst safely, escalation and closing.
- Retro-hunting and the time machine for indicators of compromise.
- User roles (admin, analyst, viewer) with least-privilege access through views and procedures.

4. Key Contacts and Stakeholders

Name	Registration Number	Role	Phone Number / Email
Shlok Gupta	24BCI0061	Team Member	
Shreya Bhushan	24BCI0086	Team Member	

Name	Registration Number	Role	Phone Number / Email
Yashneil	24BDS0158	Team Member	
Dr. Anand Bihari		Faculty Guide	

5. Project Resource Requirements

5.1 Hardware Resource Requirements

- A laptop or desktop with an Intel i5 (8th gen) or Ryzen 5 and at least 8 GB RAM.
- About 3 GB of free disk space for MySQL, the tools and the database.

5.2 Software Resource Requirements

- Windows 11 (the machine all screenshots were taken on).
- MySQL Community Server 8.0.46 with the mysql command-line client. INTERSECT and EXCEPT need 8.0.31 or later; CHECK constraints are enforced from 8.0.16.
- MySQL Workbench for browsing the schema.
- Python 3.11 with FastAPI for the backend, and pandas and scikit-learn for the ML parts.
- Node.js 20 with React and Vite for the dashboard.
- draw.io for planning the ER diagram, and Git with a shared GitHub repository.

6. System Architecture

We kept a simple layered setup with the database in the middle. Every other part either writes to it or reads from it. Logs are loaded in batches and each batch goes in as one transaction, so a failed batch leaves nothing half written (section 13 shows this). The rules that must never be skipped, like opening incidents and running retro-hunts, are triggers inside MySQL instead of application code. The FastAPI backend connects with a limited database user, and the ML jobs write their results back into existing columns.

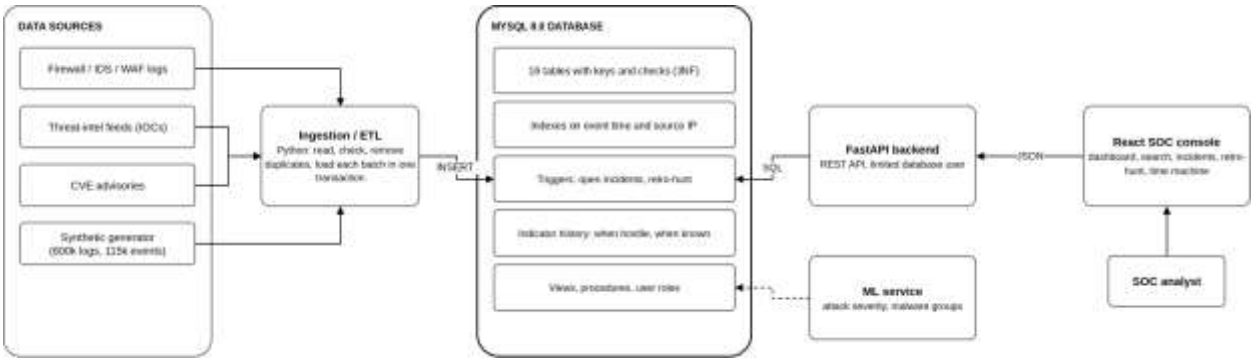


Fig. 1 System architecture

7. Module-wise Functional Description

Module	What it does	Database objects behind it
Dashboard and search	Attacks per asset, open vulnerabilities, risk, and attack search by IP, country, type, malware, CVE, severity and date	v_attack_details, v_asset_risk, fn_asset_risk, queries in section 11
Incident handling	Opens an incident for any attack of severity 8 or more, groups repeat attacks within an hour, assigns analysts safely, escalates priority	trg_attack_auto_incident, sp_assign_incident, sp_escalate_incidents
Retro-hunt	When a new bad IP is added, searches old logs from the time it was already hostile but not yet known, and opens an incident if it finds anything	trg_ioc_retro_hunt, retro_hunt_run, retro_hit, v_retro_hunt_summary
Time machine	Every indicator keeps when it was hostile and when we learned about it, so results can be split into "known then" and "learned later"	ioc.valid_from/valid_to, recorded_from/recorded_to, trg_ioc_no_overlap, trg_ioc_no_delete, CQ-05
Reports	Attack summary by type for any date range	sp_attack_report, sp_asset_risk_report
Roles and security	Admins can do everything, analysts work only on their own incidents, viewers only see reports	roles sx_admin, sx_analyst, sx_viewer; v_my_incidents

8. ER Diagram

The ER diagram on the next page is drawn in Chen notation and is unchanged from the revised Review 1. Rectangles are entities, diamonds are relationships and ovals are attributes, with the primary key underlined. MITIGATION is a weak entity (double rectangle) whose partial key step_no has a dashed underline, and its identifying relationship HAS_STEP is a double diamond. detected_at and patched_at belong to the EXPOSED_TO relationship itself. A double line means total participation, for example every attack must target an asset. There are 12 entities and 17 relationships: 12 one-to-many (one identifying), 4 many-to-many and 1 one-to-one.

Entity	What it represents
ASSET	A server or device we are protecting
VULNERABILITY	A published weakness (CVE) with its CVSS score
NETWORK_LOG	One raw line from the firewall or IDS
ATTACK_EVENT	One detected attack against an asset
ATTACK_TYPE	A kind of attack, mapped to a MITRE ATT&CK technique
MITIGATION (weak)	One response step for an attack type
THREAT_ACTOR	An attacker group
MALWARE	A piece of malicious software
IOC	An indicator of compromise: a bad IP, domain, file hash or URL
RETRO_HUNT_RUN	One automatic search of old logs for a newly added bad IP
INCIDENT	An investigation that groups related attacks
ANALYST	A person who uses the system

Relationship	Between	Type	Linked by
TARGETS	ASSET, ATTACK_EVENT	1 : N	attack_event.asset_id (NOT NULL)
CLASSIFIED_AS	ATTACK_TYPE, ATTACK_EVENT	1 : N	attack_event.type_id (NOT NULL)
ATTRIBUTED_TO	THREAT_ACTOR, ATTACK_EVENT	1 : N	attack_event.actor_id
DELIVERS	MALWARE, ATTACK_EVENT	1 : N	attack_event.malware_id
EXPLOITS	VULNERABILITY, ATTACK_EVENT	1 : N	attack_event.cve_id
EXPOSED_TO	ASSET, VULNERABILITY	M : N	link table asset_vulnerability
DESTINED_FOR	ASSET, NETWORK_LOG	1 : N	network_log.dst_asset_id (NOT NULL)
HAS_STEP	ATTACK_TYPE, MITIGATION	1 : N (identifying)	mitigation key (type_id, step_no)
DEVELOPS	THREAT_ACTOR, MALWARE	1 : N	malware.actor_id
LINKED_TO	THREAT_ACTOR, IOC	1 : N	ioc.actor_id
INDICATES	MALWARE, IOC	1 : N	ioc.malware_id
MATCHES	ATTACK_EVENT, IOC	M : N	link table event_ioc
TRIGGERS	IOC, RETRO_HUNT_RUN	1 : N	retro_hunt_run.ioc_version_id (NOT NULL)
HITS	RETRO_HUNT_RUN, NETWORK_LOG	M : N	link table retro_hit

Relationship	Between	Type	Linked by
OPENS	RETRO_HUNT_RUN, INCIDENT	1 : 1	retro_hunt_run.incident_id (UNIQUE)
GROUPS	INCIDENT, ATTACK_EVENT	M : N	link table incident_event
ASSIGNED_TO	ANALYST, INCIDENT	1 : N	incident.assigned_to

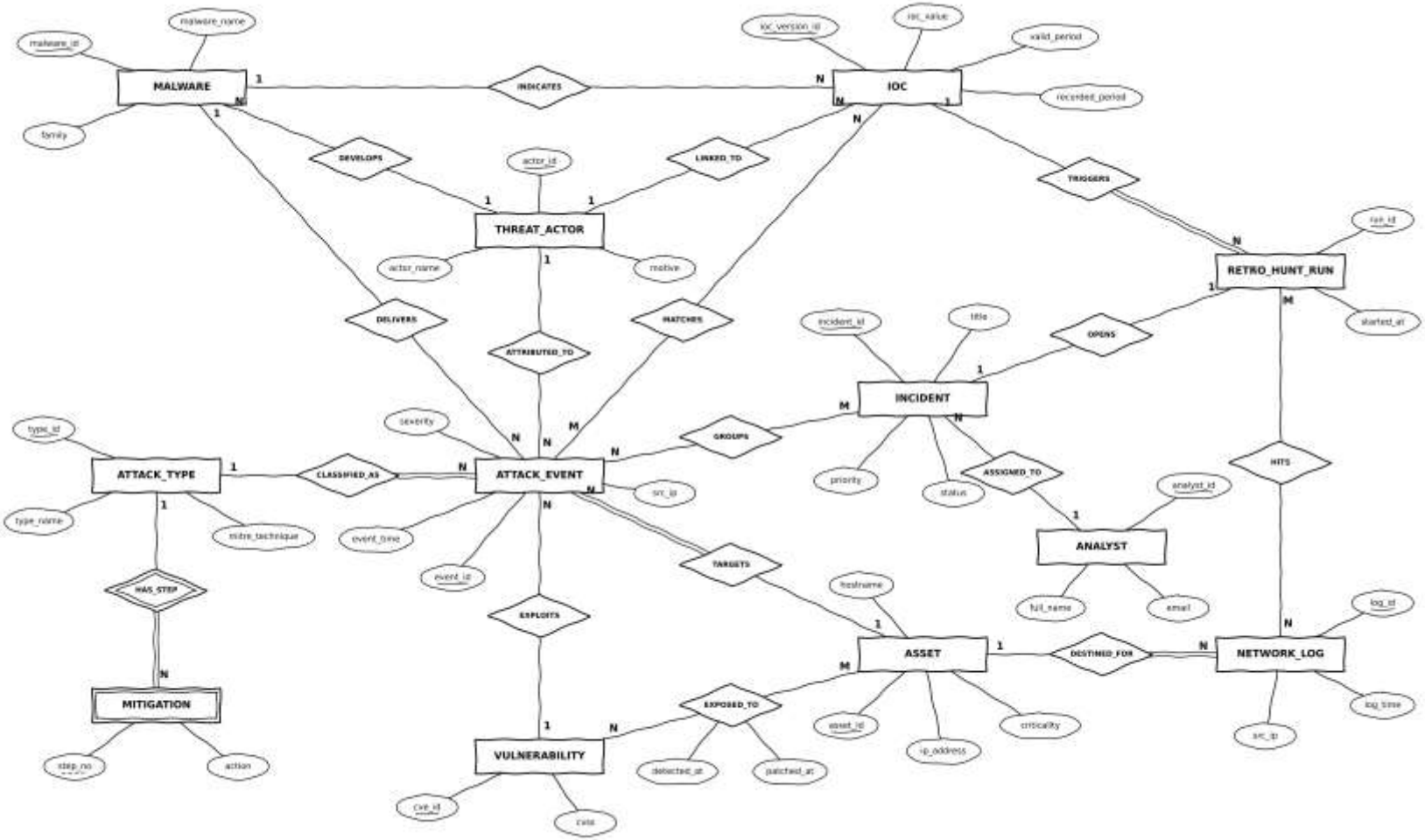


Fig. 2 ER diagram of SentinelX (Chen notation)

9. Tables and Constraints

All 16 tables are in Third Normal Form: in every table each non-key column depends on the key, the whole key and nothing but the key. Every column that decides another column is itself a candidate key, so the tables also meet BCNF. The normalisation steps and the 23 attributes removed after review are in section 15. For each table below we give a short description, its attributes with data types and constraints, the CREATE TABLE statement as MySQL ran it, and the normal form. MySQL echoes statements without comments, so the screenshots show the definitions without them.

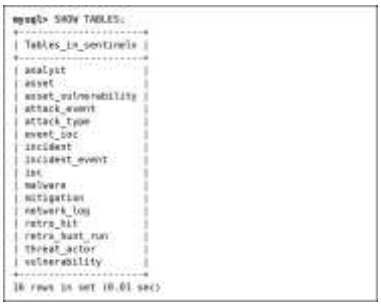


Fig. 3 SHOW TABLES after running the 16 CREATE TABLE statements

Table: asset

Every server or device we protect. hostname and ip_address are unique too.

Attribute	Data Type	Constraint
asset_id	INT	PRIMARY KEY, AUTO_INCREMENT
hostname	VARCHAR(63)	NOT NULL UNIQUE
ip_address	VARCHAR(45)	NOT NULL UNIQUE
asset_type	ENUM('web','db','mail','dns','workstation','iot','vpn')	NOT NULL
criticality	TINYINT	NOT NULL CHECK (criticality BETWEEN 1 AND 5)

```
mysql> CREATE TABLE asset (
    ->   asset_id     INT AUTO_INCREMENT PRIMARY KEY,
    ->   hostname     VARCHAR(63) NOT NULL UNIQUE,
    ->   ip_address   VARCHAR(45) NOT NULL UNIQUE,
    ->   asset_type   ENUM('web','db','mail','dns','workstation','iot','vpn') NOT NULL,
    ->   criticality  TINYINT NOT NULL CHECK (criticality BETWEEN 1 AND 5)
    -> );
Query OK, 0 rows affected (0.06 sec)
```

Fig. 4 CREATE TABLE asset executed in MySQL

Normal form: Every column depends only on asset_id (hostname and ip_address are also candidate keys). 3NF.

Table: vulnerability

Published CVEs. cve_id must look like CVE-2024-3400. Severity is not stored; the view v_vulnerability works it out from cvss.

Attribute	Data Type	Constraint
cve_id	VARCHAR(16)	PRIMARY KEY CHECK (cve_id REGEXP '^CVE-[0-9]{4}-[0-9]{4,7}$')
cvss	DECIMAL(3,1)	NOT NULL CHECK (cvss BETWEEN 0 AND 10)
published	DATE	NOT NULL

```
mysql> CREATE TABLE vulnerability (
    ->   cve_id     VARCHAR(16) PRIMARY KEY CHECK (cve_id REGEXP '^CVE-[0-9]{4}-[0-9]{4,7}$'),
    ->   cvss       DECIMAL(3,1) NOT NULL CHECK (cvss BETWEEN 0 AND 10),
    ->   published  DATE NOT NULL
    -> );
Query OK, 0 rows affected (0.04 sec)
```

Fig. 5 CREATE TABLE vulnerability executed in MySQL

Normal form: cvss and published depend only on cve_id. Storing severity would add the transitive dependency cve_id -> cvss -> severity. 3NF.

Table: asset_vulnerability

Link table for EXPOSED_TO (M:N between assets and CVEs), with when the CVE was found and patched.

Attribute	Data Type	Constraint
asset_id	INT	NOT NULL; Part of composite PRIMARY KEY (asset_id, cve_id); FOREIGN KEY to asset(asset_id), on delete cascade
cve_id	VARCHAR(16)	NOT NULL; Part of composite PRIMARY KEY (asset_id, cve_id); FOREIGN KEY to vulnerability(cve_id)
detected_at	DATE	NOT NULL; CHECK (patched_at IS NULL OR patched_at >= detected_at)
patched_at	DATE	NULL allowed

```
mysql> CREATE TABLE asset_vulnerability (
    ->   asset_id     INT NOT NULL,
    ->   cve_id       VARCHAR(16) NOT NULL,
    ->   detected_at  DATE NOT NULL,
    ->   patched_at   DATE,
    ->   PRIMARY KEY (asset_id, cve_id),
    ->   FOREIGN KEY (asset_id) REFERENCES asset(asset_id) ON DELETE CASCADE,
    ->   FOREIGN KEY (cve_id) REFERENCES vulnerability(cve_id),
    ->   CHECK (patched_at IS NULL OR patched_at >= detected_at)
    -> );
Query OK, 0 rows affected (0.05 sec)
```

Fig. 6 CREATE TABLE asset_vulnerability executed in MySQL

Normal form: detected_at and patched_at depend on the whole key (asset_id, cve_id). 3NF.

Table: network_log

Raw log lines from the firewall and IDS. The index on (src_ip, log_time) serves the retro-hunt (section 14).

Attribute	Data Type	Constraint
log_id	BIGINT	PRIMARY KEY, AUTO_INCREMENT
log_time	DATETIME	NOT NULL
src_ip	VARCHAR(45)	NOT NULL; Indexed (idx_src_time: src_ip, log_time)
dst_asset_id	INT	NOT NULL; FOREIGN KEY to asset(asset_id)
dst_port	SMALLINT UNSIGNED	NOT NULL
protocol	ENUM('TCP','UDP','ICMP')	NOT NULL
action	ENUM('allowed','blocked')	NOT NULL

```
mysql> CREATE TABLE network_log (
    ->   log_id        BIGINT AUTO_INCREMENT PRIMARY KEY,
    ->   log_time      DATETIME NOT NULL,
    ->   src_ip        VARCHAR(45) NOT NULL,
    ->   dst_asset_id  INT NOT NULL,
    ->   dst_port      SMALLINT UNSIGNED NOT NULL,
    ->   protocol      ENUM('TCP','UDP','ICMP') NOT NULL,
    ->   action        ENUM('allowed','blocked') NOT NULL,
    ->   FOREIGN KEY (dst_asset_id) REFERENCES asset(asset_id),
    ->   INDEX idx_src_time (src_ip, log_time)
    -> );
Query OK, 0 rows affected (0.05 sec)
```

Fig. 7 CREATE TABLE network_log executed in MySQL

Normal form: Everything depends on log_id. 3NF.

Table: attack_type

Kinds of attack with their MITRE ATT&CK technique ID.

Attribute	Data Type	Constraint
type_id	INT	PRIMARY KEY, AUTO_INCREMENT
type_name	VARCHAR(40)	NOT NULL UNIQUE
mitre_technique	VARCHAR(9)	NOT NULL UNIQUE CHECK (mitre_technique REGEXP '^T[0-9]{4}([.][0-9]{3})?$')
base_severity	TINYINT	NOT NULL CHECK (base_severity BETWEEN 1 AND 10)

```
mysql> CREATE TABLE attack_type (
    ->   type_id          INT AUTO_INCREMENT PRIMARY KEY,
    ->   type_name        VARCHAR(40) NOT NULL UNIQUE,
    ->   mitre_technique  VARCHAR(9) NOT NULL UNIQUE CHECK (mitre_technique REGEXP '^T[0-9]{4}([.][0-9]{3})?$'),
    ->   base_severity    TINYINT NOT NULL CHECK (base_severity BETWEEN 1 AND 10)
    -> );
Query OK, 0 rows affected (0.05 sec)
```

Fig. 8 CREATE TABLE attack_type executed in MySQL

Normal form: Everything depends on type_id; type_name and mitre_technique are candidate keys. 3NF.

Table: mitigation

Weak entity: numbered response steps for each attack type, deleted together with the type.

Attribute	Data Type	Constraint
type_id	INT	NOT NULL; Part of composite PRIMARY KEY (type_id, step_no); FOREIGN KEY to attack_type(type_id), on delete cascade
step_no	TINYINT	NOT NULL CHECK (step_no > 0); Part of composite PRIMARY KEY (type_id, step_no)
action	VARCHAR(100)	NOT NULL

```
mysql> CREATE TABLE mitigation (
    ->   type_id  INT NOT NULL,
    ->   step_no  TINYINT NOT NULL CHECK (step_no > 0),
    ->   action   VARCHAR(100) NOT NULL,
    ->   PRIMARY KEY (type_id, step_no),
    ->   FOREIGN KEY (type_id) REFERENCES attack_type(type_id) ON DELETE CASCADE
    -> );
Query OK, 0 rows affected (0.06 sec)
```

Fig. 9 CREATE TABLE mitigation executed in MySQL

Normal form: action depends on the full key (type_id, step_no). 3NF.

Table: threat_actor

Attacker groups with country and motive. The names are made up.

Attribute	Data Type	Constraint
actor_id	INT	PRIMARY KEY, AUTO_INCREMENT
actor_name	VARCHAR(50)	NOT NULL UNIQUE
origin_country	CHAR(2)	CHECK (origin_country REGEXP '^[A-Z]{2}$')
motive	ENUM('financial','espionage','hacktivism','sabotage','unknown')	NOT NULL

```
mysql> CREATE TABLE threat_actor (
    ->   actor_id        INT AUTO_INCREMENT PRIMARY KEY,
    ->   actor_name      VARCHAR(50) NOT NULL UNIQUE,
    ->   origin_country  CHAR(2) CHECK (origin_country REGEXP '^[A-Z]{2}$'),
    ->   motive          ENUM('financial','espionage','hacktivism','sabotage','unknown') NOT NULL
    -> );
Query OK, 0 rows affected (0.06 sec)
```

Fig. 10 CREATE TABLE threat_actor executed in MySQL

Normal form: Everything depends on actor_id. 3NF.

Table: malware

Malware samples. actor_id becomes NULL if the group is deleted; cluster_id is filled by the ML job.

Attribute	Data Type	Constraint
malware_id	INT	PRIMARY KEY, AUTO_INCREMENT

Attribute	Data Type	Constraint
malware_name	VARCHAR(50)	NOT NULL UNIQUE
family	VARCHAR(30)	NOT NULL
sha256	CHAR(64)	NOT NULL UNIQUE CHECK (sha256 REGEXP '^[0-9a-f]{64}$')
actor_id	INT	FOREIGN KEY to threat_actor(actor_id), on delete set null
cluster_id	SMALLINT	NULL allowed

```
mysql> CREATE TABLE malware (
    ->   malware_id    INT AUTO_INCREMENT PRIMARY KEY,
    ->   malware_name  VARCHAR(50) NOT NULL UNIQUE,
    ->   family        VARCHAR(30) NOT NULL,
    ->   sha256        CHAR(64) NOT NULL UNIQUE CHECK (sha256 REGEXP '^[0-9a-f]{64}$'),
    ->   actor_id      INT,
    ->   cluster_id    SMALLINT,
    ->   FOREIGN KEY (actor_id) REFERENCES threat_actor(actor_id) ON DELETE SET NULL
    -> );
Query OK, 0 rows affected (0.06 sec)
```

Fig. 11 CREATE TABLE malware executed in MySQL

Normal form: Everything depends on malware_id. category was removed because family decides it. 3NF.

Table: attack_event

One row per detected attack. Asset and type are required; attacker, malware and CVE are optional.

Attribute	Data Type	Constraint
event_id	BIGINT	PRIMARY KEY, AUTO_INCREMENT
event_time	DATETIME	NOT NULL; Indexed (idx_time: event_time)
src_ip	VARCHAR(45)	NOT NULL; Indexed (idx_src: src_ip)
src_country	CHAR(2)	NOT NULL CHECK (src_country REGEXP '^[A-Z]{2}$')
asset_id	INT	NOT NULL; FOREIGN KEY to asset(asset_id)
type_id	INT	NOT NULL; FOREIGN KEY to attack_type(type_id)
actor_id	INT	FOREIGN KEY to threat_actor(actor_id)
malware_id	INT	FOREIGN KEY to malware(malware_id)
cve_id	VARCHAR(16)	FOREIGN KEY to vulnerability(cve_id)
severity	TINYINT	NOT NULL CHECK (severity BETWEEN 1 AND 10)
blocked	BOOLEAN	NOT NULL
predicted_severity	ENUM('LOW','MEDIUM','HIGH','CRITICAL')	NULL allowed

```
mysql> CREATE TABLE attack_event (
    ->   event_id           BIGINT AUTO_INCREMENT PRIMARY KEY,
    ->   event_time         DATETIME NOT NULL,
    ->   src_ip             VARCHAR(45) NOT NULL,
    ->   src_country        CHAR(2) NOT NULL CHECK (src_country REGEXP '^[A-Z]{2}$'),
    ->   asset_id           INT NOT NULL,
    ->   type_id            INT NOT NULL,
    ->   actor_id           INT,
    ->   malware_id         INT,
    ->   cve_id             VARCHAR(16),
    ->   severity           TINYINT NOT NULL CHECK (severity BETWEEN 1 AND 10),
    ->   blocked            BOOLEAN NOT NULL,
    ->   predicted_severity  ENUM('LOW','MEDIUM','HIGH','CRITICAL'),
    ->   FOREIGN KEY (asset_id) REFERENCES asset(asset_id),
    ->   FOREIGN KEY (type_id) REFERENCES attack_type(type_id),
    ->   FOREIGN KEY (actor_id) REFERENCES threat_actor(actor_id),
    ->   FOREIGN KEY (malware_id) REFERENCES malware(malware_id),
    ->   FOREIGN KEY (cve_id) REFERENCES vulnerability(cve_id),
    ->   INDEX idx_time (event_time),
    ->   INDEX idx_src (src_ip)
    -> );
Query OK, 0 rows affected (0.10 sec)
```

Fig. 12 CREATE TABLE attack_event executed in MySQL

Normal form: Everything depends on event_id. src_country stays because an IP's location changes over time; actor_id stays because malware is shared between groups. 3NF.

Table: analyst

People who use the system. email is unique and checked to look like an email address.

Attribute	Data Type	Constraint
analyst_id	INT	PRIMARY KEY, AUTO_INCREMENT
full_name	VARCHAR(50)	NOT NULL
email	VARCHAR(50)	NOT NULL UNIQUE CHECK (email REGEXP '^[^@]+@[^@]+[.][A-Za-z]{2,}$')
role	ENUM('admin','analyst','viewer')	NOT NULL

```
mysql> CREATE TABLE analyst (
    ->   analyst_id  INT AUTO_INCREMENT PRIMARY KEY,
    ->   full_name   VARCHAR(50) NOT NULL,
    ->   email       VARCHAR(50) NOT NULL UNIQUE CHECK (email REGEXP '^[^@ ]+@[^@ ]+[.][A-Za-z]{2,}$'),
    ->   role        ENUM('admin','analyst','viewer') NOT NULL
    -> );
Query OK, 0 rows affected (0.07 sec)
```

Fig. 13 CREATE TABLE analyst executed in MySQL

Normal form: Everything depends on analyst_id; email is also a candidate key. 3NF.

Table: incident

An investigation. closed_at is filled in only when the status is CLOSED.

Attribute	Data Type	Constraint
incident_id	INT	PRIMARY KEY, AUTO_INCREMENT
title	VARCHAR(100)	NOT NULL

Attribute	Data Type	Constraint
priority	ENUM('LOW','MEDIUM','HIGH','CRITICAL')	NOT NULL
status	ENUM('OPEN','INVESTIGATING','CONTAINED','CLOSED')	NOT NULL DEFAULT 'OPEN'; CHECK ((status = 'CLOSED') = (closed_at IS NOT NULL))
assigned_to	INT	FOREIGN KEY to analyst(analyst_id)
opened_at	DATETIME	NOT NULL DEFAULT CURRENT_TIMESTAMP; CHECK (closed_at IS NULL OR closed_at >= opened_at)
closed_at	DATETIME	NULL allowed

```
mysql> CREATE TABLE incident (
    ->   incident_id  INT AUTO_INCREMENT PRIMARY KEY,
    ->   title        VARCHAR(100) NOT NULL,
    ->   priority     ENUM('LOW','MEDIUM','HIGH','CRITICAL') NOT NULL,
    ->   status       ENUM('OPEN','INVESTIGATING','CONTAINED','CLOSED') NOT NULL DEFAULT 'OPEN',
    ->   assigned_to  INT,
    ->   opened_at    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ->   closed_at    DATETIME,
    ->   FOREIGN KEY (assigned_to) REFERENCES analyst(analyst_id),
    ->   CHECK ((status = 'CLOSED') = (closed_at IS NOT NULL)),
    ->   CHECK (closed_at IS NULL OR closed_at >= opened_at)
    -> );
Query OK, 0 rows affected (0.05 sec)
```

Fig. 14 CREATE TABLE incident executed in MySQL

Normal form: Everything depends on incident_id. 3NF.

Table: incident_event

Link table for GROUPS (M:N between incidents and attacks).

Attribute	Data Type	Constraint
incident_id	INT	NOT NULL; Part of composite PRIMARY KEY (incident_id, event_id); FOREIGN KEY to incident(incident_id), on delete cascade
event_id	BIGINT	NOT NULL; Part of composite PRIMARY KEY (incident_id, event_id); FOREIGN KEY to attack_event(event_id), on delete cascade

```
mysql> CREATE TABLE incident_event (
    ->   incident_id  INT NOT NULL,
    ->   event_id     BIGINT NOT NULL,
    ->   PRIMARY KEY (incident_id, event_id),
    ->   FOREIGN KEY (incident_id) REFERENCES incident(incident_id) ON DELETE CASCADE,
    ->   FOREIGN KEY (event_id) REFERENCES attack_event(event_id) ON DELETE CASCADE
    -> );
Query OK, 0 rows affected (0.13 sec)
```

Fig. 15 CREATE TABLE incident_event executed in MySQL

Normal form: All-key table. 3NF.

Table: ioc

Indicators of compromise. valid_from/valid_to say when it was hostile and recorded_from/recorded_to say when we knew. Triggers keep the history append-only.

Attribute	Data Type	Constraint
ioc_version_id	BIGINT	PRIMARY KEY, AUTO_INCREMENT
ioc_type	ENUM('ip','domain','hash','url')	NOT NULL; CHECK (ioc_type <> 'ip' OR ioc_value REGEXP '^[0-9]{1,3}([.][0-9]{1,3}){3}$')
ioc_value	VARCHAR(255)	NOT NULL; Indexed (idx_value: ioc_value)
actor_id	INT	FOREIGN KEY to threat_actor(actor_id)
malware_id	INT	FOREIGN KEY to malware(malware_id)
valid_from	DATETIME	NOT NULL; CHECK (valid_to IS NULL OR valid_to > valid_from)
valid_to	DATETIME	NULL allowed
recorded_from	DATETIME	NOT NULL DEFAULT CURRENT_TIMESTAMP; CHECK (recorded_to IS NULL OR recorded_to > recorded_from)
recorded_to	DATETIME	NULL allowed

```
mysql> CREATE TABLE ioc (
    ->   ioc_version_id  BIGINT AUTO_INCREMENT PRIMARY KEY,
    ->   ioc_type        ENUM('ip','domain','hash','url') NOT NULL,
    ->   ioc_value       VARCHAR(255) NOT NULL,
    ->   actor_id        INT,
    ->   malware_id      INT,
    ->   valid_from      DATETIME NOT NULL,
    ->   valid_to        DATETIME,
    ->   recorded_from   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ->   recorded_to     DATETIME,
    ->   FOREIGN KEY (actor_id) REFERENCES threat_actor(actor_id),
    ->   FOREIGN KEY (malware_id) REFERENCES malware(malware_id),
    ->   CHECK (ioc_type <> 'ip' OR ioc_value REGEXP '^[0-9]{1,3}([.][0-9]{1,3}){3}$'),
    ->   CHECK (valid_to IS NULL OR valid_to > valid_from),
    ->   CHECK (recorded_to IS NULL OR recorded_to > recorded_from),
    ->   INDEX idx_value (ioc_value)
    -> );
Query OK, 0 rows affected (0.06 sec)
```

Fig. 16 CREATE TABLE ioc executed in MySQL

Normal form: Everything depends on ioc_version_id. 3NF.

Table: event_ioc

Link table for MATCHES (M:N between attacks and indicators).

Attribute	Data Type	Constraint
event_id	BIGINT	NOT NULL; Part of composite PRIMARY KEY (event_id, ioc_version_id); FOREIGN KEY to attack_event(event_id), on delete cascade
ioc_version_id	BIGINT	NOT NULL; Part of composite PRIMARY KEY (event_id, ioc_version_id); FOREIGN KEY to ioc(ioc_version_id)

```
mysql> CREATE TABLE event_ioc (
    ->   event_id        BIGINT NOT NULL,
    ->   ioc_version_id  BIGINT NOT NULL,
    ->   PRIMARY KEY (event_id, ioc_version_id),
    ->   FOREIGN KEY (event_id) REFERENCES attack_event(event_id) ON DELETE CASCADE,
    ->   FOREIGN KEY (ioc_version_id) REFERENCES ioc(ioc_version_id)
    -> );
Query OK, 0 rows affected (0.07 sec)
```

Fig. 17 CREATE TABLE event_ioc executed in MySQL

Normal form: All-key table. 3NF.

Table: retro_hunt_run

One row per automatic search of old logs. incident_id is UNIQUE (1:1 OPENS).

Attribute	Data Type	Constraint
run_id	INT	PRIMARY KEY, AUTO_INCREMENT
ioc_version_id	BIGINT	NOT NULL; FOREIGN KEY to ioc(ioc_version_id)
started_at	DATETIME	NOT NULL DEFAULT CURRENT_TIMESTAMP
incident_id	INT	UNIQUE; FOREIGN KEY to incident(incident_id)

```
mysql> CREATE TABLE retro_hunt_run (
    ->   run_id          INT AUTO_INCREMENT PRIMARY KEY,
    ->   ioc_version_id  BIGINT NOT NULL,
    ->   started_at      DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ->   incident_id     INT UNIQUE,
    ->   FOREIGN KEY (ioc_version_id) REFERENCES ioc(ioc_version_id),
    ->   FOREIGN KEY (incident_id) REFERENCES incident(incident_id)
    -> );
Query OK, 0 rows affected (0.06 sec)
```

Fig. 18 CREATE TABLE retro_hunt_run executed in MySQL

Normal form: Everything depends on run_id; counts come from v_retro_hunt_summary instead of being stored. 3NF.

Table: retro_hit

Link table for HITS: the old log lines a retro-hunt found.

Attribute	Data Type	Constraint
run_id	INT	NOT NULL; Part of composite PRIMARY KEY (run_id, log_id); FOREIGN KEY to retro_hunt_run(run_id), on delete cascade
log_id	BIGINT	NOT NULL; Part of composite PRIMARY KEY (run_id, log_id); FOREIGN KEY to network_log(log_id)

```
mysql> CREATE TABLE retro_hit (
    ->   run_id  INT NOT NULL,
    ->   log_id  BIGINT NOT NULL,
    ->   PRIMARY KEY (run_id, log_id),
    ->   FOREIGN KEY (run_id) REFERENCES retro_hunt_run(run_id) ON DELETE CASCADE,
    ->   FOREIGN KEY (log_id) REFERENCES network_log(log_id)
    -> );
Query OK, 0 rows affected (0.05 sec)
```

Fig. 19 CREATE TABLE retro_hit executed in MySQL

Normal form: All-key table. 3NF.

10. Data Population (INSERT, UPDATE, DELETE)

The data is a hand-made test set, small enough to check every result by hand, plus 50,000 generated background log lines so the index comparison in section 14 is meaningful. Attacker IPs come from the address ranges reserved for documentation (192.0.2.0/24, 198.51.100.0/24, 203.0.113.0/24), so no real address is marked as bad. The CVE IDs are real. The full script is 02_dml.sql; one INSERT per table type is shown here.

INSERT

```
mysql> INSERT INTO asset (asset_id, hostname, ip_address, asset_type, criticality) VALUES
    ->   (1, 'web-01', '10.0.1.10', 'web', 5),
    ->   (2, 'web-02', '10.0.1.11', 'web', 4),
    ->   (3, 'db-01', '10.0.2.20', 'db', 5),
    ->   (4, 'mail-01', '10.0.3.30', 'mail', 3),
    ->   (5, 'dns-01', '10.0.4.40', 'dns', 4),
    ->   (6, 'ws-hr-01', '10.0.5.51', 'workstation', 2),
    ->   (7, 'ws-fin-01', '10.0.5.52', 'workstation', 3),
    ->   (8, 'cam-01', '10.0.6.60', 'iot', 1),
    ->   (9, 'vpn-01', '10.0.7.70', 'vpn', 4),
    ->   (10, 'ws-dev-01', '10.0.5.53', 'workstation', 2);
Query OK, 10 rows affected (0.01 sec)
Records: 10  Duplicates: 0  Warnings: 0
```

Fig. 20 INSERT of the 10 assets

The other 14 tables are filled the same way, with explicit keys so the links between tables are predictable. The background log lines are generated with a recursive CTE in one statement:

```
mysql> SET SESSION cte_max_recursion_depth = 60000;
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO network_log (log_time, src_ip, dst_asset_id, dst_port, protocol, action)
    -> WITH RECURSIVE seq (n) AS (SELECT 1 UNION ALL SELECT n + 1 FROM seq WHERE n < 50000)
    -> SELECT TIMESTAMP('2026-09-01') + INTERVAL n * 100 SECOND,
    ->        CONCAT('172.16.', n MOD 40, '.', 1 + n MOD 250),
    ->        1 + n MOD 9,
    ->        ELT(1 + n MOD 5, 443, 80, 53, 25, 22),
    ->        IF(n MOD 5 = 2, 'UDP', 'TCP'),
    ->        IF(n MOD 47 = 0, 'blocked', 'allowed')
    -> FROM seq;
Query OK, 50000 rows affected (5.12 sec)
Records: 50000  Duplicates: 0  Warnings: 0
```

Fig. 21 Generating 50,000 background log lines with a recursive CTE

UPDATE

The ML job had not scored attacks 25 and 26. The UPDATE sets their predicted severity from the same bands the job uses.

```
mysql> SELECT event_id, severity, predicted_severity FROM attack_event WHERE event_id IN (25, 26);
+----------+----------+--------------------+
| event_id | severity | predicted_severity |
+----------+----------+--------------------+
|       25 |        7 | NULL               |
|       26 |        4 | NULL               |
+----------+----------+--------------------+
2 rows in set (0.00 sec)

mysql> UPDATE attack_event
    ->    SET predicted_severity = CASE WHEN severity >= 9 THEN 'CRITICAL'
    ->                                  WHEN severity >= 7 THEN 'HIGH'
    ->                                  WHEN severity >= 4 THEN 'MEDIUM'
    ->                                  ELSE 'LOW' END
    ->  WHERE event_id IN (25, 26);
Query OK, 2 rows affected (0.02 sec)
Rows matched: 2  Changed: 2  Warnings: 0

mysql> SELECT event_id, severity, predicted_severity FROM attack_event WHERE event_id IN (25, 26);
+----------+----------+--------------------+
| event_id | severity | predicted_severity |
+----------+----------+--------------------+
|       25 |        7 | HIGH               |
|       26 |        4 | MEDIUM             |
+----------+----------+--------------------+
2 rows in set (0.00 sec)
```

Fig. 22 UPDATE of predicted_severity, before and after

DELETE

```
mysql> INSERT INTO asset (hostname, ip_address, asset_type, criticality) VALUES ('tmp-scan-01', '10.0.9.99', 'workstation', 1);
Query OK, 1 row affected (0.01 sec)

mysql> SELECT asset_id, hostname FROM asset WHERE hostname = 'tmp-scan-01';
+----------+-------------+
| asset_id | hostname    |
+----------+-------------+
|       11 | tmp-scan-01 |
+----------+-------------+
1 row in set (0.00 sec)

mysql> DELETE FROM asset WHERE hostname = 'tmp-scan-01';
Query OK, 1 row affected (0.00 sec)

mysql> SELECT COUNT(*) AS assets_left FROM asset;
+-------------+
| assets_left |
+-------------+
|          10 |
+-------------+
1 row in set (0.00 sec)
```

Fig. 23 INSERT and DELETE of a temporary asset

Constraints in action

Each statement below breaks one rule on purpose, and MySQL rejects it: a CHECK (criticality 9), a UNIQUE (duplicate hostname), a FOREIGN KEY on insert (type 99 does not exist), a FOREIGN KEY on delete (web-01 still has attacks), and the table-level CHECK that only closed incidents have a closed_at.

```
mysql> INSERT INTO asset (hostname, ip_address, asset_type, criticality) VALUES ('web-03', '10.0.1.12', 'web', 9);
ERROR 3819 (HY000): Check constraint 'asset_chk_1' is violated.

mysql> INSERT INTO asset (hostname, ip_address, asset_type, criticality) VALUES ('web-01', '10.0.1.99', 'web', 3);
ERROR 1062 (23000): Duplicate entry 'web-01' for key 'asset.hostname'

mysql> INSERT INTO attack_event (event_time, src_ip, src_country, asset_id, type_id, severity, blocked)
    -> VALUES ('2026-10-27 10:00:00', '203.0.113.5', 'US', 1, 99, 5, 1);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails
  (`sentinelx`.`attack_event`, CONSTRAINT `attack_event_ibfk_2` FOREIGN KEY (`type_id`) REFERENCES
  `attack_type` (`type_id`))

mysql> DELETE FROM asset WHERE asset_id = 1;
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails
  (`sentinelx`.`attack_event`, CONSTRAINT `attack_event_ibfk_1` FOREIGN KEY (`asset_id`) REFERENCES `asset`
  (`asset_id`))

mysql> INSERT INTO incident (title, priority, status, closed_at) VALUES ('Test', 'LOW', 'OPEN', '2026-10-27 10:00:00');
ERROR 3819 (HY000): Check constraint 'incident_chk_1' is violated.
```

Fig. 24 Five rejected statements, one per constraint

Row counts

```
mysql> SELECT 'asset' AS table_name, COUNT(*) AS row_count FROM asset
    -> UNION ALL SELECT 'vulnerability', COUNT(*) FROM vulnerability
    -> UNION ALL SELECT 'asset_vulnerability', COUNT(*) FROM asset_vulnerability
    -> UNION ALL SELECT 'network_log', COUNT(*) FROM network_log
    -> UNION ALL SELECT 'attack_type', COUNT(*) FROM attack_type
    -> UNION ALL SELECT 'mitigation', COUNT(*) FROM mitigation
    -> UNION ALL SELECT 'threat_actor', COUNT(*) FROM threat_actor
    -> UNION ALL SELECT 'malware', COUNT(*) FROM malware
    -> UNION ALL SELECT 'attack_event', COUNT(*) FROM attack_event
    -> UNION ALL SELECT 'analyst', COUNT(*) FROM analyst
    -> UNION ALL SELECT 'incident', COUNT(*) FROM incident
    -> UNION ALL SELECT 'incident_event', COUNT(*) FROM incident_event
    -> UNION ALL SELECT 'ioc', COUNT(*) FROM ioc
    -> UNION ALL SELECT 'event_ioc', COUNT(*) FROM event_ioc
    -> UNION ALL SELECT 'retro_hunt_run', COUNT(*) FROM retro_hunt_run
    -> UNION ALL SELECT 'retro_hit', COUNT(*) FROM retro_hit;
+---------------------+-----------+
| table_name          | row_count |
+---------------------+-----------+
| asset               |        10 |
| vulnerability       |         9 |
| asset_vulnerability |         9 |
| network_log         |     50021 |
| attack_type         |         8 |
| mitigation          |        18 |
| threat_actor        |         6 |
| malware             |         8 |
| attack_event        |        30 |
| analyst             |         6 |
| incident            |         7 |
| incident_event      |        18 |
| ioc                 |        12 |
| event_ioc           |        27 |
| retro_hunt_run      |         2 |
| retro_hit           |         8 |
+---------------------+-----------+
16 rows in set (0.05 sec)
```

Fig. 25 Row count of every table after loading

11. SQL Implementation

Every query below was run on the loaded database, and each screenshot shows the statement together with its real output. The table shows how the 40 queries cover what the review requires.

Category	Required coverage	Min	Done	Query IDs
A. Basic queries	SELECT, WHERE, ORDER BY, DISTINCT, BETWEEN, IN, LIKE, IS NULL	6	7	A1, A2, A3, A4, A5, A6, A7
B. Aggregate queries	COUNT, SUM, AVG, MIN, MAX, GROUP BY, HAVING (WHERE vs HAVING in B5)	5	6	B1, B2, B3, B4, B5, B6
C. Joins	INNER, LEFT, RIGHT, FULL OUTER (emulated with UNION), SELF, CROSS	6	6	C1, C2, C3, C4, C5, C6
D. Subqueries	Scalar, single-row, multi-row (IN, ANY, ALL), correlated, EXISTS, NOT EXISTS, NOT IN with the NULL pitfall	6	9	D1, D2, D3, D4, D5, D6, D7, D8, D9
E. Set operations	UNION, UNION ALL, INTERSECT, EXCEPT	4	4	E1, E2, E3, E4
F. Advanced functions	CASE, COALESCE, NULLIF, date/time, string and numeric functions	6	6	F1, F2, F3, F4, F5, F6
G. Window functions	RANK, DENSE_RANK, ROW_NUMBER, running total, LAG, LEAD	2	3	G1, G2, G3

11.1 A. Basic queries

A1. Critical assets (criticality 4 or 5), most critical first

```
mysql> SELECT hostname, ip_address, asset_type, criticality
    -> FROM asset
    -> WHERE criticality >= 4
    -> ORDER BY criticality DESC, hostname;
+----------+------------+------------+-------------+
| hostname | ip_address | asset_type | criticality |
+----------+------------+------------+-------------+
| db-01    | 10.0.2.20  | db         |           5 |
| web-01   | 10.0.1.10  | web        |           5 |
| dns-01   | 10.0.4.40  | dns        |           4 |
| vpn-01   | 10.0.7.70  | vpn        |           4 |
| web-02   | 10.0.1.11  | web        |           4 |
+----------+------------+------------+-------------+
5 rows in set (0.00 sec)
```

Fig. 26 A1: Critical assets (criticality 4 or 5), most critical first

A2. Countries that attacks came from (DISTINCT)

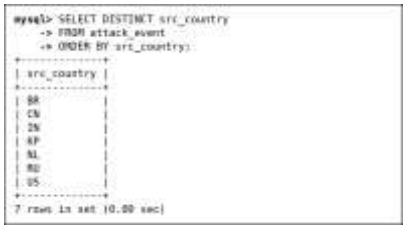


Fig. 27 A2: Countries that attacks came from (DISTINCT)

A3. Attacks between 10 and 14 October (BETWEEN)

```
mysql> SELECT event_id, event_time, src_ip, severity
    -> FROM attack_event
    -> WHERE event_time BETWEEN '2026-10-10 00:00:00' AND '2026-10-14 23:59:59'
    -> ORDER BY event_time;
+----------+---------------------+---------------+----------+
| event_id | event_time          | src_ip        | severity |
+----------+---------------------+---------------+----------+
|       15 | 2026-10-10 08:15:00 | 203.0.113.11  |       10 |
|       16 | 2026-10-10 08:40:00 | 203.0.113.11  |        9 |
|       17 | 2026-10-11 12:00:00 | 198.51.100.23 |        5 |
|       18 | 2026-10-12 19:30:00 | 203.0.113.99  |        4 |
|       19 | 2026-10-13 07:05:00 | 198.51.100.88 |        6 |
|       20 | 2026-10-14 23:50:00 | 192.0.2.50    |        6 |
+----------+---------------------+---------------+----------+
6 rows in set (0.00 sec)
```

Fig. 28 A3: Attacks between 10 and 14 October (BETWEEN)

A4. Internet-facing servers: web, db and vpn assets (IN)

```
mysql> SELECT hostname, asset_type, criticality
    -> FROM asset
    -> WHERE asset_type IN ('web', 'db', 'vpn')
    -> ORDER BY asset_type, hostname;
+----------+------------+-------------+
| hostname | asset_type | criticality |
+----------+------------+-------------+
| web-01   | web        |           5 |
| web-02   | web        |           4 |
| db-01    | db         |           5 |
| vpn-01   | vpn        |           4 |
+----------+------------+-------------+
4 rows in set (0.00 sec)
```

Fig. 29 A4: Internet-facing servers: web, db and vpn assets (IN)

A5. CVEs published in 2023 (LIKE)

```
mysql> SELECT cve_id, cvss, published
    -> FROM vulnerability
    -> WHERE cve_id LIKE 'CVE-2023-%'
    -> ORDER BY cvss DESC;
+---------------+------+------------+
| cve_id        | cvss | published  |
+---------------+------+------------+
| CVE-2023-34362 |  9.8 | 2023-06-02 |
| CVE-2023-4966  |  9.4 | 2023-10-10 |
| CVE-2023-44487 |  7.5 | 2023-10-10 |
| CVE-2023-48795 |  5.9 | 2023-12-18 |
+---------------+------+------------+
4 rows in set (0.00 sec)
```

Fig. 30 A5: CVEs published in 2023 (LIKE)

A6. Incidents nobody owns yet (IS NULL)

```
mysql> SELECT incident_id, title, priority, status
    -> FROM incident
    -> WHERE assigned_to IS NULL;
+-------------+-----------------------------------------------+----------+--------+
| incident_id | title                                         | priority | status |
+-------------+-----------------------------------------------+----------+--------+
|           5 | Retro-hunt: 203.0.113.11 hit 2 assets 4 times | CRITICAL | OPEN   |
+-------------+-----------------------------------------------+----------+--------+
1 row in set (0.00 sec)
```

Fig. 31 A6: Incidents nobody owns yet (IS NULL)

A7. Attacks the ML job has not scored yet (IS NULL)

```
mysql> SELECT event_id, event_time, src_ip, severity
    -> FROM attack_event
    -> WHERE predicted_severity IS NULL
    -> ORDER BY event_id;
+----------+---------------------+---------------+----------+
| event_id | event_time          | src_ip        | severity |
+----------+---------------------+---------------+----------+
|       27 | 2026-10-22 21:30:00 | 198.51.100.88 |        5 |
|       28 | 2026-10-24 03:15:00 | 203.0.113.11  |        7 |
|       29 | 2026-10-25 11:11:00 | 198.51.100.23 |        9 |
|       30 | 2026-10-26 15:40:00 | 192.0.2.50    |       10 |
+----------+---------------------+---------------+----------+
4 rows in set (0.00 sec)
```

Fig. 32 A7: Attacks the ML job has not scored yet (IS NULL)

11.2 B. Aggregate queries

B1. Number of attacks per attack type (COUNT, GROUP BY)

```
mysql> SELECT t.type_name, COUNT(*) AS attacks
    -> FROM attack_event e
    -> JOIN attack_type t ON t.type_id = e.type_id
    -> GROUP BY t.type_name
    -> ORDER BY attacks DESC, t.type_name;
+--------------------------+---------+
| type_name                | attacks |
+--------------------------+---------+
| Exploit Public-Facing App |       7 |
| Brute Force              |       6 |
| DDoS                     |       5 |
| Phishing                 |       4 |
| Port Scan                |       4 |
| Credential Dumping       |       2 |
| Ransomware               |       2 |
+--------------------------+---------+
7 rows in set (0.00 sec)
```

Fig. 33 B1: Number of attacks per attack type (COUNT, GROUP BY)

B2. Attacks, blocked attacks and average severity per asset (SUM, AVG)

```
mysql> SELECT a.hostname,
    ->        COUNT(*)                  AS attacks,
    ->        SUM(e.blocked)            AS blocked,
    ->        ROUND(AVG(e.severity), 2) AS avg_severity
    -> FROM attack_event e
    -> JOIN asset a ON a.asset_id = e.asset_id
    -> GROUP BY a.hostname
    -> ORDER BY attacks DESC, a.hostname;
+-----------+---------+---------+--------------+
| hostname  | attacks | blocked | avg_severity |
+-----------+---------+---------+--------------+
| web-01    |       7 |       3 |         6.43 |
| web-02    |       5 |       3 |         7.80 |
| db-01     |       4 |       2 |         9.00 |
| vpn-01    |       4 |       2 |         7.75 |
| mail-01   |       3 |       1 |         5.67 |
| cam-01    |       2 |       1 |         4.00 |
| dns-01    |       2 |       1 |         4.50 |
| ws-fin-01 |       2 |       2 |         5.50 |
| ws-hr-01  |       1 |       1 |         5.00 |
+-----------+---------+---------+--------------+
9 rows in set (0.00 sec)
```

Fig. 34 B2: Attacks, blocked attacks and average severity per asset (SUM, AVG)

B3. First and last attack and worst severity per source country (MIN, MAX)

```
mysql> SELECT src_country,
    ->        MIN(event_time) AS first_seen,
    ->        MAX(event_time) AS last_seen,
    ->        MAX(severity)   AS worst_severity
    -> FROM attack_event
    -> GROUP BY src_country
    -> ORDER BY first_seen;
+-------------+---------------------+---------------------+----------------+
| src_country | first_seen          | last_seen           | worst_severity |
+-------------+---------------------+---------------------+----------------+
| RU          | 2026-10-01 02:14:00 | 2026-10-25 11:11:00 |             10 |
| US          | 2026-10-02 11:05:00 | 2026-10-16 13:13:00 |              3 |
| CN          | 2026-10-03 09:30:00 | 2026-10-24 03:15:00 |             10 |
| KP          | 2026-10-05 01:00:00 | 2026-10-26 15:40:00 |             10 |
| BR          | 2026-10-06 14:20:00 | 2026-10-22 21:30:00 |              7 |
| NL          | 2026-10-09 16:45:00 | 2026-10-20 18:00:00 |              8 |
| IN          | 2026-10-12 19:30:00 | 2026-10-21 09:09:00 |              4 |
+-------------+---------------------+---------------------+----------------+
7 rows in set (0.00 sec)
```

Fig. 35 B3: First and last attack and worst severity per source country (MIN, MAX)

B4. Source IPs with more than 3 attacks (HAVING)

```
mysql> SELECT src_ip, COUNT(*) AS attacks
    -> FROM attack_event
    -> GROUP BY src_ip
    -> HAVING COUNT(*) > 3
    -> ORDER BY attacks DESC, src_ip;
+--------------+---------+
| src_ip       | attacks |
+--------------+---------+
| 192.0.2.50   |       4 |
| 198.51.100.23 |       4 |
| 198.51.100.45 |       4 |
| 198.51.100.88 |       4 |
| 203.0.113.200 |       4 |
+--------------+---------+
5 rows in set (0.00 sec)
```

Fig. 36 B4: Source IPs with more than 3 attacks (HAVING)

B5. WHERE vs HAVING: assets with 2 or more serious attacks

WHERE removes attacks below severity 7 before any grouping happens; HAVING then removes whole groups (assets) that end up with fewer than 2 rows. Moving the severity filter into HAVING would not work, because severity is not a group-level value.

```
mysql> SELECT a.hostname, COUNT(*) AS serious_attacks
    -> FROM attack_event e
    -> JOIN asset a ON a.asset_id = e.asset_id
    -> WHERE e.severity >= 7
    -> GROUP BY a.hostname
    -> HAVING COUNT(*) >= 2
    -> ORDER BY serious_attacks DESC, a.hostname;
+----------+-----------------+
| hostname | serious_attacks |
+----------+-----------------+
| web-02   |               5 |
| db-01    |               4 |
| web-01   |               4 |
| vpn-01   |               3 |
+----------+-----------------+
4 rows in set (0.00 sec)
```

Fig. 37 B5: WHERE vs HAVING: assets with 2 or more serious attacks

B6. CVSS statistics by year of publication

```
mysql> SELECT YEAR(published)       AS year_published,
    ->        COUNT(*)              AS cves,
    ->        ROUND(AVG(cvss), 2)   AS avg_cvss,
    ->        MIN(cvss)             AS min_cvss,
    ->        MAX(cvss)             AS max_cvss
    -> FROM vulnerability
    -> GROUP BY YEAR(published)
    -> ORDER BY year_published;
+----------------+------+----------+----------+----------+
| year_published | cves | avg_cvss | min_cvss | max_cvss |
+----------------+------+----------+----------+----------+
|           2019 |    1 |     9.80 |      9.8 |      9.8 |
|           2021 |    1 |    10.00 |     10.0 |     10.0 |
|           2022 |    1 |     9.80 |      9.8 |      9.8 |
|           2023 |    4 |     8.15 |      5.9 |      9.8 |
|           2024 |    2 |     9.05 |      8.1 |     10.0 |
+----------------+------+----------+----------+----------+
5 rows in set (0.00 sec)
```

Fig. 38 B6: CVSS statistics by year of publication

11.3 C. Joins

C1. INNER JOIN: attacks of severity 9 or more with asset and attack type

```
mysql> SELECT e.event_id, e.event_time, a.hostname, t.type_name, e.severity
    -> FROM attack_event e
    -> INNER JOIN asset a       ON a.asset_id = e.asset_id
    -> INNER JOIN attack_type t ON t.type_id  = e.type_id
    -> WHERE e.severity >= 9
    -> ORDER BY e.event_time;
+----------+---------------------+----------+--------------------------+----------+
| event_id | event_time          | hostname | type_name                | severity |
+----------+---------------------+----------+--------------------------+----------+
|        5 | 2026-10-04 22:10:00 | mail-01  | Exploit Public-Facing App |        9 |
|        6 | 2026-10-05 01:00:00 | db-01    | Exploit Public-Facing App |        9 |
|        7 | 2026-10-05 01:45:00 | db-01    | Credential Dumping        |        9 |
|       11 | 2026-10-07 04:10:00 | web-02   | Ransomware                |       10 |
|       15 | 2026-10-10 08:15:00 | vpn-01   | Exploit Public-Facing App |       10 |
|       16 | 2026-10-10 08:40:00 | vpn-01   | Credential Dumping        |        9 |
|       29 | 2026-10-25 11:11:00 | vpn-01   | Exploit Public-Facing App |        9 |
|       30 | 2026-10-26 15:40:00 | db-01    | Ransomware                |       10 |
+----------+---------------------+----------+--------------------------+----------+
8 rows in set (0.00 sec)
```

Fig. 39 C1: INNER JOIN: attacks of severity 9 or more with asset and attack type

C2. LEFT JOIN: every asset with its number of attacks, including assets never attacked

```
mysql> SELECT a.hostname, COUNT(e.event_id) AS attacks
    -> FROM asset a
    -> LEFT JOIN attack_event e ON e.asset_id = a.asset_id
    -> GROUP BY a.hostname
    -> ORDER BY attacks, a.hostname;
+-----------+---------+
| hostname  | attacks |
+-----------+---------+
| ws-dev-01 |       0 |
| ws-hr-01  |       1 |
| cam-01    |       2 |
| dns-01    |       2 |
| ws-fin-01 |       2 |
| mail-01   |       3 |
| db-01     |       4 |
| vpn-01    |       4 |
| web-02    |       5 |
| web-01    |       7 |
+-----------+---------+
10 rows in set (0.00 sec)
```

Fig. 40 C2: LEFT JOIN: every asset with its number of attacks, including assets never attacked

C3. RIGHT JOIN: every analyst with the incidents they own, including analysts with none

```
mysql> SELECT an.full_name, an.role, i.incident_id, i.status
    -> FROM incident i
    -> RIGHT JOIN analyst an ON an.analyst_id = i.assigned_to
    -> ORDER BY an.analyst_id, i.incident_id;
+-------------+---------+-------------+---------------+
| full_name   | role    | incident_id | status        |
+-------------+---------+-------------+---------------+
| Ananya Rao  | admin   |        NULL | NULL          |
| Rahul Mehta | analyst |           1 | CLOSED        |
| Rahul Mehta | analyst |           3 | CONTAINED     |
| Rahul Mehta | analyst |           7 | INVESTIGATING |
| Priya Nair  | analyst |           2 | CLOSED        |
| Priya Nair  | analyst |           6 | OPEN          |
| Karan Shah  | analyst |           4 | INVESTIGATING |
| Divya Iyer  | viewer  |        NULL | NULL          |
| Arjun Das   | analyst |        NULL | NULL          |
+-------------+---------+-------------+---------------+
9 rows in set (0.00 sec)
```

Fig. 41 C3: RIGHT JOIN: every analyst with the incidents they own, including analysts with none

C4. FULL OUTER JOIN emulation: threat actors and malware, keeping both unmatched sides

MySQL has no FULL OUTER JOIN, so it is emulated as LEFT JOIN UNION RIGHT JOIN. The result keeps Byte Bandits (no malware) and OrphanMiner (no known actor).

```
mysql> SELECT t.actor_name, m.malware_name
    -> FROM threat_actor t LEFT JOIN malware m ON m.actor_id = t.actor_id
    -> UNION
    -> SELECT t.actor_name, m.malware_name
    -> FROM threat_actor t RIGHT JOIN malware m ON m.actor_id = t.actor_id
    -> ORDER BY actor_name, malware_name;
+----------------+----------------+
| actor_name     | malware_name   |
+----------------+----------------+
| NULL           | OrphanMiner    |
| Byte Bandits   | NULL           |
| Crimson Lynx   | LynxLoader     |
| Crimson Lynx   | LynxLocker     |
| Ghost Parrot   | ParrotBot      |
| Iron Kestrel   | KestrelWiper   |
| Quiet Willow   | WillowRAT      |
| Silent Monsoon | MonsoonRAT     |
| Silent Monsoon | MonsoonStealer |
+----------------+----------------+
9 rows in set (0.00 sec)
```

Fig. 42 C4: FULL OUTER JOIN emulation: threat actors and malware, keeping both unmatched sides

C5. SELF JOIN: pairs of malware that the ML job put in the same cluster

```
mysql> SELECT m1.cluster_id, m1.malware_name AS malware_1, m2.malware_name AS malware_2
    -> FROM malware m1
    -> JOIN malware m2 ON m2.cluster_id = m1.cluster_id
    ->                AND m2.malware_id > m1.malware_id
    -> ORDER BY m1.cluster_id, malware_1;
+------------+------------+----------------+
| cluster_id | malware_1  | malware_2      |
+------------+------------+----------------+
|          1 | LynxLocker | KestrelWiper   |
|          2 | LynxLoader | MonsoonStealer |
|          3 | MonsoonRAT | WillowRAT      |
|          4 | ParrotBot  | OrphanMiner    |
+------------+------------+----------------+
4 rows in set (0.00 sec)
```

Fig. 43 C5: SELF JOIN: pairs of malware that the ML job put in the same cluster

C6. CROSS JOIN: hardening checklist of every criticality-5 asset against every severe attack type

A meaningful CROSS JOIN: every criticality-5 asset is paired with every attack type of base severity 8 or more, to produce a hardening checklist. Zero means the combination has not been seen yet.

```
mysql> SELECT a.hostname, t.type_name,
    ->        (SELECT COUNT(*) FROM attack_event e
    ->          WHERE e.asset_id = a.asset_id AND e.type_id = t.type_id) AS times_seen
    -> FROM asset a
    -> CROSS JOIN attack_type t
    -> WHERE a.criticality = 5 AND t.base_severity >= 8
    -> ORDER BY a.hostname, t.type_name;
+----------+--------------------------+------------+
| hostname | type_name                | times_seen |
+----------+--------------------------+------------+
| db-01    | Credential Dumping       |          1 |
| db-01    | Exploit Public-Facing App |          2 |
| db-01    | Ransomware               |          1 |
| web-01   | Credential Dumping       |          0 |
| web-01   | Exploit Public-Facing App |          0 |
| web-01   | Ransomware               |          0 |
+----------+--------------------------+------------+
6 rows in set (0.00 sec)
```

Fig. 44 C6: CROSS JOIN: hardening checklist of every criticality-5 asset against every severe attack type

11.4 D. Subqueries

D1. Scalar subquery: CVEs scoring above the average CVSS

```
mysql> SELECT cve_id, cvss,
    ->        (SELECT ROUND(AVG(cvss), 2) FROM vulnerability) AS avg_cvss
    -> FROM vulnerability
    -> WHERE cvss > (SELECT AVG(cvss) FROM vulnerability)
    -> ORDER BY cvss DESC, cve_id;
+----------------+------+----------+
| cve_id         | cvss | avg_cvss |
+----------------+------+----------+
| CVE-2021-44228 | 10.0 |     8.92 |
| CVE-2024-3400  | 10.0 |     8.92 |
| CVE-2019-0708  |  9.8 |     8.92 |
| CVE-2022-22965 |  9.8 |     8.92 |
| CVE-2023-34362 |  9.8 |     8.92 |
| CVE-2023-4966  |  9.4 |     8.92 |
+----------------+------+----------+
6 rows in set (0.00 sec)
```

Fig. 45 D1: Scalar subquery: CVEs scoring above the average CVSS

D2. Single-row subquery: the analyst who owns the most recently opened incident

```
mysql> SELECT full_name, email
    -> FROM analyst
    -> WHERE analyst_id = (SELECT assigned_to
    ->                     FROM incident
    ->                     WHERE assigned_to IS NOT NULL
    ->                     ORDER BY opened_at DESC
    ->                     LIMIT 1);
+------------+-------------------------+
| full_name  | email                   |
+------------+-------------------------+
| Priya Nair | priya.nair@sentinelx.in |
+------------+-------------------------+
1 row in set (0.00 sec)
```

Fig. 46 D2: Single-row subquery: the analyst who owns the most recently opened incident

D3. Multi-row subquery with IN: assets that have been hit by ransomware

```
mysql> SELECT hostname, asset_type
    -> FROM asset
    -> WHERE asset_id IN (SELECT e.asset_id
    ->                    FROM attack_event e
    ->                    JOIN attack_type t ON t.type_id = e.type_id
    ->                    WHERE t.type_name = 'Ransomware')
    -> ORDER BY hostname;
+----------+------------+
| hostname | asset_type |
+----------+------------+
| db-01    | db         |
| web-02   | web        |
+----------+------------+
2 rows in set (0.00 sec)
```

Fig. 47 D3: Multi-row subquery with IN: assets that have been hit by ransomware

D4. Multi-row subquery with ALL: CVEs scoring higher than every CVE on web-02 and mail-01

```
mysql> SELECT cve_id, cvss
    -> FROM vulnerability
    -> WHERE cvss > ALL (SELECT v.cvss
    ->                   FROM asset_vulnerability av
    ->                   JOIN vulnerability v ON v.cve_id = av.cve_id
    ->                   WHERE av.asset_id IN (2, 4))
    -> ORDER BY cve_id;
+----------------+------+
| cve_id         | cvss |
+----------------+------+
| CVE-2021-44228 | 10.0 |
| CVE-2024-3400  | 10.0 |
+----------------+------+
2 rows in set (0.00 sec)
```

Fig. 48 D4: Multi-row subquery with ALL: CVEs scoring higher than every CVE on web-02 and mail-01

D5. Multi-row subquery with ANY: CVEs scoring higher than at least one CVE used by Russian sources

```
mysql> SELECT cve_id, cvss
    -> FROM vulnerability
    -> WHERE cvss > ANY (SELECT v.cvss
    ->                   FROM attack_event e
    ->                   JOIN vulnerability v ON v.cve_id = e.cve_id
    ->                   WHERE e.src_country = 'RU')
    -> ORDER BY cvss DESC, cve_id;
+----------------+------+
| cve_id         | cvss |
+----------------+------+
| CVE-2021-44228 | 10.0 |
| CVE-2024-3400  | 10.0 |
| CVE-2019-0708  |  9.8 |
| CVE-2022-22965 |  9.8 |
| CVE-2023-34362 |  9.8 |
| CVE-2023-4966  |  9.4 |
+----------------+------+
6 rows in set (0.00 sec)
```

Fig. 49 D5: Multi-row subquery with ANY: CVEs scoring higher than at least one CVE used by Russian sources

D6. Correlated subquery: the worst attack(s) on each asset

```
mysql> SELECT a.hostname, e.event_id, e.severity
    -> FROM attack_event e
    -> JOIN asset a ON a.asset_id = e.asset_id
    -> WHERE e.severity = (SELECT MAX(e2.severity)
    ->                     FROM attack_event e2
    ->                     WHERE e2.asset_id = e.asset_id)
    -> ORDER BY a.hostname, e.event_id;
+-----------+----------+----------+
| hostname  | event_id | severity |
+-----------+----------+----------+
| cam-01    |       18 |        4 |
| cam-01    |       26 |        4 |
| db-01     |       30 |       10 |
| dns-01    |       28 |        7 |
| mail-01   |        5 |        9 |
| vpn-01    |       15 |       10 |
| web-01    |       14 |        8 |
| web-01    |       24 |        8 |
| web-02    |       11 |       10 |
| ws-fin-01 |       20 |        6 |
| ws-hr-01  |       17 |        5 |
+-----------+----------+----------+
11 rows in set (0.00 sec)
```

Fig. 50 D6: Correlated subquery: the worst attack(s) on each asset

D7. EXISTS: threat actors with at least one attack that was not blocked

```
mysql> SELECT t.actor_name, t.motive
    -> FROM threat_actor t
    -> WHERE EXISTS (SELECT 1 FROM attack_event e
    ->               WHERE e.actor_id = t.actor_id AND e.blocked = 0)
    -> ORDER BY t.actor_name;
+----------------+------------+
| actor_name     | motive     |
+----------------+------------+
| Byte Bandits   | hacktivism |
| Crimson Lynx   | financial  |
| Iron Kestrel   | sabotage   |
| Silent Monsoon | espionage  |
+----------------+------------+
4 rows in set (0.00 sec)
```

Fig. 51 D7: EXISTS: threat actors with at least one attack that was not blocked

D8. NOT EXISTS: assets that have never been attacked

```
mysql> SELECT hostname, asset_type, criticality
    -> FROM asset a
    -> WHERE NOT EXISTS (SELECT 1 FROM attack_event e
    ->                   WHERE e.asset_id = a.asset_id);
+-----------+-------------+-------------+
| hostname  | asset_type  | criticality |
+-----------+-------------+-------------+
| ws-dev-01 | workstation |           2 |
+-----------+-------------+-------------+
1 row in set (0.00 sec)
```

Fig. 52 D8: NOT EXISTS: assets that have never been attacked

D9. NOT IN and the NULL pitfall: threat actors with no malware

The NULL pitfall: malware.actor_id contains a NULL (OrphanMiner). `x NOT IN (..., NULL)` is never TRUE, so the first query returns no rows at all. Filtering out NULLs in the subquery (or using NOT EXISTS) gives the correct answer, Byte Bandits.

```
mysql> SELECT actor_name FROM threat_actor
    -> WHERE actor_id NOT IN (SELECT actor_id FROM malware);
Empty set (0.00 sec)

mysql> SELECT actor_name FROM threat_actor
    -> WHERE actor_id NOT IN (SELECT actor_id FROM malware WHERE actor_id IS NOT NULL);
+--------------+
| actor_name   |
+--------------+
| Byte Bandits |
+--------------+
1 row in set (0.00 sec)
```

Fig. 53 D9: NOT IN and the NULL pitfall: threat actors with no malware

11.5 E. Set operations

E1. UNION: every country we know about, as an attack source or as an attacker's origin

```
mysql> SELECT src_country AS country FROM attack_event
    -> UNION
    -> SELECT origin_country FROM threat_actor WHERE origin_country IS NOT NULL
    -> ORDER BY country;
+---------+
| country |
+---------+
| BR      |
| CN      |
| IN      |
| IR      |
| KP      |
| NL      |
| RU      |
| US      |
+---------+
8 rows in set (0.00 sec)
```

Fig. 54 E1: UNION: every country we know about, as an attack source or as an attacker's origin

E2. UNION ALL: full activity timeline of 198.51.100.45 across attacks and raw logs

```
mysql> SELECT event_time AS seen_at, 'attack' AS source, asset_id, severity AS detail
    -> FROM attack_event WHERE src_ip = '198.51.100.45'
    -> UNION ALL
    -> SELECT log_time, 'log', dst_asset_id, dst_port
    -> FROM network_log WHERE src_ip = '198.51.100.45'
    -> ORDER BY seen_at;
+---------------------+--------+----------+--------+
| seen_at             | source | asset_id | detail |
+---------------------+--------+----------+--------+
| 2026-10-03 03:00:00 | log    |        2 |   8080 |
| 2026-10-05 03:10:00 | log    |        2 |   8080 |
| 2026-10-06 23:00:00 | log    |        1 |    443 |
| 2026-10-07 03:30:00 | log    |        2 |   8080 |
| 2026-10-07 03:33:00 | attack |        2 |      8 |
| 2026-10-07 04:10:00 | attack |        2 |     10 |
| 2026-10-18 02:01:00 | log    |        1 |     22 |
| 2026-10-18 02:02:00 | attack |        1 |      7 |
| 2026-10-18 02:03:00 | log    |        1 |     22 |
| 2026-10-18 02:05:00 | attack |        1 |      8 |
+---------------------+--------+----------+--------+
10 rows in set (0.00 sec)
```

Fig. 55 E2: UNION ALL: full activity timeline of 198.51.100.45 across attacks and raw logs

E3. INTERSECT: attack sources that are also known bad IPs

INTERSECT and EXCEPT are native in MySQL 8.0.31 and later; we ran them on 8.0.46.

```
mysql> SELECT src_ip FROM attack_event
    -> INTERSECT
    -> SELECT ioc_value FROM ioc WHERE ioc_type = 'ip'
    -> ORDER BY src_ip;
+--------------+
| src_ip       |
+--------------+
| 192.0.2.50   |
| 192.0.2.77   |
| 198.51.100.23 |
| 198.51.100.45 |
| 198.51.100.88 |
| 203.0.113.10  |
| 203.0.113.11  |
+--------------+
7 rows in set (0.00 sec)
```

Fig. 56 E3: INTERSECT: attack sources that are also known bad IPs

E4. EXCEPT: attack sources that are not on the indicator list yet

```
mysql> SELECT src_ip FROM attack_event
    -> EXCEPT
    -> SELECT ioc_value FROM ioc WHERE ioc_type = 'ip'
    -> ORDER BY src_ip;
+--------------+
| src_ip       |
+--------------+
| 203.0.113.200 |
| 203.0.113.99  |
+--------------+
2 rows in set (0.00 sec)
```

Fig. 57 E4: EXCEPT: attack sources that are not on the indicator list yet

11.6 F. Advanced functions

F1. CASE: severity label from CVSS score

```
mysql> SELECT cve_id, cvss,
    ->        CASE WHEN cvss >= 9 THEN 'CRITICAL'
    ->             WHEN cvss >= 7 THEN 'HIGH'
    ->             WHEN cvss >= 4 THEN 'MEDIUM'
    ->             ELSE 'LOW' END AS severity
    -> FROM vulnerability
    -> ORDER BY cvss DESC, cve_id;
+----------------+------+----------+
| cve_id         | cvss | severity |
+----------------+------+----------+
| CVE-2021-44228 | 10.0 | CRITICAL |
| CVE-2024-3400  | 10.0 | CRITICAL |
| CVE-2019-0708  |  9.8 | CRITICAL |
| CVE-2022-22965 |  9.8 | CRITICAL |
| CVE-2023-34362 |  9.8 | CRITICAL |
| CVE-2023-4966  |  9.4 | CRITICAL |
| CVE-2024-6387  |  8.1 | HIGH     |
| CVE-2023-44487 |  7.5 | HIGH     |
| CVE-2023-48795 |  5.9 | MEDIUM   |
+----------------+------+----------+
9 rows in set (0.00 sec)
```

Fig. 58 F1: CASE: severity label from CVSS score

F2. COALESCE: recent attacks with 'Unattributed' instead of a missing actor

```
mysql> SELECT e.event_id, e.src_ip,
    ->        COALESCE(t.actor_name, 'Unattributed') AS actor,
    ->        COALESCE(m.malware_name, '-')          AS malware
    -> FROM attack_event e
    -> LEFT JOIN threat_actor t ON t.actor_id   = e.actor_id
    -> LEFT JOIN malware m      ON m.malware_id = e.malware_id
    -> WHERE e.event_time >= '2026-10-20'
    -> ORDER BY e.event_id;
+----------+--------------+----------------+----------------+
| event_id | src_ip       | actor          | malware        |
+----------+--------------+----------------+----------------+
|       25 | 192.0.2.77   | Byte Bandits   | -              |
|       26 | 203.0.113.99 | Unattributed   | OrphanMiner    |
|       27 | 198.51.100.88| Ghost Parrot   | ParrotBot      |
|       28 | 203.0.113.11 | Silent Monsoon | MonsoonStealer |
|       29 | 198.51.100.23| Crimson Lynx   | LynxLoader     |
|       30 | 192.0.2.50   | Iron Kestrel   | KestrelWiper   |
+----------+--------------+----------------+----------------+
6 rows in set (0.00 sec)
```

Fig. 59 F2: COALESCE: recent attacks with 'Unattributed' instead of a missing actor

F3. NULLIF: share of blocked attacks per asset without dividing by zero

NULLIF turns a zero count into NULL, so ws-dev-01 (never attacked) shows NULL instead of a division-by-zero.

```
mysql> SELECT a.hostname,
    ->        COUNT(e.event_id) AS attacks,
    ->        ROUND(100 * SUM(e.blocked) / NULLIF(COUNT(e.event_id), 0), 1) AS blocked_pct
    -> FROM asset a
    -> LEFT JOIN attack_event e ON e.asset_id = a.asset_id
    -> GROUP BY a.hostname
    -> ORDER BY blocked_pct DESC, a.hostname;
+-----------+---------+-------------+
| hostname  | attacks | blocked_pct |
+-----------+---------+-------------+
| ws-fin-01 |       2 |       100.0 |
| ws-hr-01  |       1 |       100.0 |
| web-02    |       5 |        60.0 |
| cam-01    |       2 |        50.0 |
| db-01     |       4 |        50.0 |
| dns-01    |       2 |        50.0 |
| vpn-01    |       4 |        50.0 |
| web-01    |       7 |        42.9 |
| mail-01   |       3 |        33.3 |
| ws-dev-01 |       0 |        NULL |
+-----------+---------+-------------+
10 rows in set (0.00 sec)
```

Fig. 60 F3: NULLIF: share of blocked attacks per asset without dividing by zero

F4. Date and time functions: how long each incident has been or was open

```
mysql> SELECT incident_id,
    ->        DATE_FORMAT(opened_at, '%d %b %Y %H:%i') AS opened,
    ->        DAYNAME(opened_at) AS opened_on,
    ->        TIMESTAMPDIFF(HOUR, opened_at, COALESCE(closed_at, '2026-10-28 12:00:00')) AS hours_open,
    ->        DATEDIFF('2026-10-28', DATE(opened_at)) AS age_days
    -> FROM incident
    -> ORDER BY incident_id;
+-------------+------------------+-----------+------------+----------+
| incident_id | opened           | opened_on | hours_open | age_days |
+-------------+------------------+-----------+------------+----------+
|           1 | 01 Oct 2026 02:25 | Thursday  |          7 |       27 |
|           2 | 04 Oct 2026 22:20 | Sunday    |         43 |       24 |
|           3 | 05 Oct 2026 01:10 | Monday    |        562 |       23 |
|           4 | 12 Oct 2026 08:00 | Monday    |        388 |       16 |
|           5 | 20 Oct 2026 09:00 | Tuesday   |        195 |        8 |
|           6 | 20 Oct 2026 18:10 | Tuesday   |        185 |        8 |
|           7 | 18 Oct 2026 02:10 | Sunday    |        249 |       10 |
+-------------+------------------+-----------+------------+----------+
7 rows in set (0.00 sec)
```

Fig. 61 F4: Date and time functions: how long each incident has been or was open

F5. String functions: analyst user names and short names

```
mysql> SELECT LPAD(analyst_id, 4, '0')                    AS staff_no,
    ->        UPPER(full_name)                             AS name_upper,
    ->        CONCAT(LEFT(full_name, 1), '. ',
    ->               SUBSTRING_INDEX(full_name, ' ', -1))  AS short_name,
    ->        SUBSTRING_INDEX(email, '@', 1)               AS username,
    ->        LENGTH(email)                                AS email_length
    -> FROM analyst
    -> ORDER BY analyst_id;
+----------+------------+------------+-------------+--------------+
| staff_no | name_upper | short_name | username    | email_length |
+----------+------------+------------+-------------+--------------+
| 0001     | ANANYA RAO | A. Rao     | ananya.rao  |           23 |
| 0002     | RAHUL MEHTA| R. Mehta   | rahul.mehta |           24 |
| 0003     | PRIYA NAIR | P. Nair    | priya.nair  |           23 |
| 0004     | KARAN SHAH | K. Shah    | karan.shah  |           23 |
| 0005     | DIVYA IYER | D. Iyer    | divya.iyer  |           23 |
| 0006     | ARJUN DAS  | A. Das     | arjun.das   |           22 |
+----------+------------+------------+-------------+--------------+
6 rows in set (0.00 sec)
```

Fig. 62 F5: String functions: analyst user names and short names

F6. Numeric functions on CVSS scores

```
mysql> SELECT cve_id, cvss,
    ->        ROUND(cvss)              AS rounded,
    ->        CEIL(cvss)               AS ceiling,
    ->        FLOOR(cvss)              AS floor_value,
    ->        TRUNCATE(cvss * 10, 0)   AS pct_of_max,
    ->        MOD(FLOOR(cvss * 10), 7) AS bucket
    -> FROM vulnerability
    -> ORDER BY cvss DESC, cve_id;
+----------------+------+---------+---------+-------------+------------+--------+
| cve_id         | cvss | rounded | ceiling | floor_value | pct_of_max | bucket |
+----------------+------+---------+---------+-------------+------------+--------+
| CVE-2021-44228 | 10.0 |      10 |      10 |          10 |        100 |      2 |
| CVE-2024-3400  | 10.0 |      10 |      10 |          10 |        100 |      2 |
| CVE-2019-0708  |  9.8 |      10 |      10 |           9 |         98 |      0 |
| CVE-2022-22965 |  9.8 |      10 |      10 |           9 |         98 |      0 |
| CVE-2023-34362 |  9.8 |      10 |      10 |           9 |         98 |      0 |
| CVE-2023-4966  |  9.4 |       9 |      10 |           9 |         94 |      3 |
| CVE-2024-6387  |  8.1 |       8 |       9 |           8 |         81 |      4 |
| CVE-2023-44487 |  7.5 |       8 |       8 |           7 |         75 |      5 |
| CVE-2023-48795 |  5.9 |       6 |       6 |           5 |         59 |      3 |
+----------------+------+---------+---------+-------------+------------+--------+
9 rows in set (0.00 sec)
```

Fig. 63 F6: Numeric functions on CVSS scores

11.7 G. Window functions

G1. RANK, DENSE_RANK and ROW_NUMBER: assets ranked by number of attacks

RANK leaves gaps after ties (3, 3, 5), DENSE_RANK does not (3, 3, 4), and ROW_NUMBER is always unique.

```
mysql> SELECT a.hostname,
    ->        COUNT(e.event_id) AS attacks,
    ->        RANK()       OVER (ORDER BY COUNT(e.event_id) DESC) AS rnk,
    ->        DENSE_RANK() OVER (ORDER BY COUNT(e.event_id) DESC) AS dense_rnk,
    ->        ROW_NUMBER() OVER (ORDER BY COUNT(e.event_id) DESC, a.hostname) AS row_no
    -> FROM asset a
    -> LEFT JOIN attack_event e ON e.asset_id = a.asset_id
    -> GROUP BY a.hostname
    -> ORDER BY row_no;
+-----------+---------+-----+-----------+--------+
| hostname  | attacks | rnk | dense_rnk | row_no |
+-----------+---------+-----+-----------+--------+
| web-01    |       7 |   1 |         1 |      1 |
| web-02    |       5 |   2 |         2 |      2 |
| db-01     |       4 |   3 |         3 |      3 |
| vpn-01    |       4 |   3 |         3 |      4 |
| mail-01   |       3 |   5 |         4 |      5 |
| cam-01    |       2 |   6 |         5 |      6 |
| dns-01    |       2 |   6 |         5 |      7 |
| ws-fin-01 |       2 |   6 |         5 |      8 |
| ws-hr-01  |       1 |   9 |         6 |      9 |
| ws-dev-01 |       0 |  10 |         7 |     10 |
+-----------+---------+-----+-----------+--------+
10 rows in set (0.00 sec)
```

Fig. 64 G1: RANK, DENSE_RANK and ROW_NUMBER: assets ranked by number of attacks

G2. Running total of attacks per day for the first ten days of October

```
mysql> SELECT DATE(event_time) AS day,
    ->        COUNT(*) AS attacks,
    ->        SUM(COUNT(*)) OVER (ORDER BY DATE(event_time)) AS running_total
    -> FROM attack_event
    -> WHERE event_time < '2026-10-11'
    -> GROUP BY DATE(event_time)
    -> ORDER BY day;
+------------+---------+---------------+
| day        | attacks | running_total |
+------------+---------+---------------+
| 2026-10-01 |       2 |             2 |
| 2026-10-02 |       1 |             3 |
| 2026-10-03 |       1 |             4 |
| 2026-10-04 |       1 |             5 |
| 2026-10-05 |       2 |             7 |
| 2026-10-06 |       2 |             9 |
| 2026-10-07 |       2 |            11 |
| 2026-10-08 |       2 |            13 |
| 2026-10-09 |       1 |            14 |
| 2026-10-10 |       2 |            16 |
+------------+---------+---------------+
10 rows in set (0.00 sec)
```

Fig. 65 G2: Running total of attacks per day for the first ten days of October

G3. LAG and LEAD: minutes between attacks from the same Russian source IP

```
mysql> SELECT src_ip, event_id, event_time,
    ->        TIMESTAMPDIFF(MINUTE,
    ->                      LAG(event_time) OVER w, event_time) AS mins_since_prev,
    ->        LEAD(event_id) OVER w AS next_event
    -> FROM attack_event
    -> WHERE src_country = 'RU'
    -> WINDOW w AS (PARTITION BY src_ip ORDER BY event_time)
    -> ORDER BY src_ip, event_time;
+---------------+----------+---------------------+-----------------+------------+
| src_ip        | event_id | event_time          | mins_since_prev | next_event |
+---------------+----------+---------------------+-----------------+------------+
| 198.51.100.23 |        1 | 2026-10-01 02:14:00 |            NULL |          2 |
| 198.51.100.23 |        2 | 2026-10-01 02:20:00 |               6 |         17 |
| 198.51.100.23 |       17 | 2026-10-11 12:00:00 |           14980 |         29 |
| 198.51.100.23 |       29 | 2026-10-25 11:11:00 |           20111 |       NULL |
| 198.51.100.45 |       10 | 2026-10-07 03:33:00 |            NULL |         11 |
| 198.51.100.45 |       11 | 2026-10-07 04:10:00 |              37 |         23 |
| 198.51.100.45 |       23 | 2026-10-18 02:02:00 |           15712 |         24 |
| 198.51.100.45 |       24 | 2026-10-18 02:05:00 |               3 |       NULL |
+---------------+----------+---------------------+-----------------+------------+
8 rows in set (0.00 sec)
```

Fig. 66 G3: LAG and LEAD: minutes between attacks from the same Russian source IP

12. Advanced SQL Implementation

12.1 Complex Business Queries

Each query uses at least three tables or a nested or analytic construct. The expected output was worked out by hand from the test data before running; the actual output is the screenshot. All 11 matched.

CQ-01

Field	Content
ID	CQ-01
Business question	Which threat actor is behind the most serious attacks (severity 8+) on our important assets (criticality 4+)?

SQL	`SELECT t.actor_name, t.motive,`
	`COUNT(*) AS serious_attacks,`
	`COUNT(DISTINCT e.asset_id) AS assets_hit,`
	`MAX(e.severity) AS worst`
	`FROM attack_event e`
	`JOIN threat_actor t ON t.actor_id = e.actor_id`
	`JOIN asset a ON a.asset_id = e.asset_id`
	`WHERE e.severity >= 8 AND a.criticality >= 4`
	`GROUP BY t.actor_id, t.actor_name, t.motive`
	`ORDER BY serious_attacks DESC, t.actor_name;`

Expected output	4 rows. Crimson Lynx first with 4 serious attacks on 3 assets (events 10, 11, 24, 29); Iron Kestrel 3 on db-01; Silent Monsoon 3 on 2 assets; Byte Bandits 1. Ghost Parrot is absent because its worst attack is severity 7.
Actual output screenshot	Below (Fig. 67)
Explanation	Three tables: attack_event joined to threat_actor (drops unattributed attacks) and to asset. WHERE keeps severity 8+ on criticality 4+ assets before grouping by actor; COUNT(DISTINCT asset_id) counts assets, not attacks.

```
mysql> SELECT t.actor_name, t.motive,
    ->        COUNT(*)                  AS serious_attacks,
    ->        COUNT(DISTINCT e.asset_id) AS assets_hit,
    ->        MAX(e.severity)           AS worst
    -> FROM attack_event e
    -> JOIN threat_actor t ON t.actor_id = e.actor_id
    -> JOIN asset a        ON a.asset_id = e.asset_id
    -> WHERE e.severity >= 8 AND a.criticality >= 4
    -> GROUP BY t.actor_id, t.actor_name, t.motive
    -> ORDER BY serious_attacks DESC, t.actor_name;
+---------------+-----------+-----------------+------------+-------+
| actor_name    | motive    | serious_attacks | assets_hit | worst |
+---------------+-----------+-----------------+------------+-------+
| Crimson Lynx  | financial |               4 |          3 |    10 |
| Iron Kestrel  | sabotage  |               3 |          1 |    10 |
| Silent Monsoon| espionage |               3 |          2 |    10 |
| Byte Bandits  | hacktivism|               1 |          1 |     8 |
+---------------+-----------+-----------------+------------+-------+
4 rows in set (0.00 sec)
```

Fig. 67 CQ-01 actual output

CQ-02

Field	Content
ID	CQ-02
Business question	What does the board of incidents that are still not closed look like?

SQL	`SELECT i.incident_id, i.priority, i.status,`
	`COALESCE(an.full_name, 'Unassigned') AS owner,`
	`COUNT(*) AS attacks,`
	`MAX(e.severity) AS worst,`
	`GROUP_CONCAT(DISTINCT a.hostname ORDER BY a.hostname) AS assets`
	`FROM incident i`
	`JOIN incident_event ie ON ie.incident_id = i.incident_id`
	`JOIN attack_event e ON e.event_id = ie.event_id`
	`JOIN asset a ON a.asset_id = e.asset_id`
	`LEFT JOIN analyst an ON an.analyst_id = i.assigned_to`
	`WHERE i.status <> 'CLOSED'`
	`GROUP BY i.incident_id, i.priority, i.status, an.full_name`
	`ORDER BY FIELD(i.priority, 'CRITICAL', 'HIGH', 'MEDIUM', 'LOW'),`
	`i.incident_id;`

Expected output	5 incidents (all except the closed 1 and 2): 4 and 5 CRITICAL, 3 and 7 HIGH, 6 MEDIUM. Incident 5 has no owner; incident 6 covers web-01 and web-02 with 5 attacks.
Actual output screenshot	Below (Fig. 68)
Explanation	Five tables. incident to incident_event to attack_event to asset gives the attacks and hosts; LEFT JOIN analyst keeps the unassigned incident and COALESCE prints "Unassigned". FIELD() sorts by priority order instead of alphabetically.

```
mysql> SELECT i.incident_id, i.priority, i.status,
    ->        COALESCE(an.full_name, 'Unassigned')      AS owner,
    ->        COUNT(*)                                  AS attacks,
    ->        MAX(e.severity)                           AS worst,
    ->        GROUP_CONCAT(DISTINCT a.hostname ORDER BY a.hostname) AS assets
    -> FROM incident i
    -> JOIN incident_event ie ON ie.incident_id = i.incident_id
    -> JOIN attack_event e    ON e.event_id     = ie.event_id
    -> JOIN asset a           ON a.asset_id     = e.asset_id
    -> LEFT JOIN analyst an   ON an.analyst_id  = i.assigned_to
    -> WHERE i.status <> 'CLOSED'
    -> GROUP BY i.incident_id, i.priority, i.status, an.full_name
    -> ORDER BY FIELD(i.priority, 'CRITICAL', 'HIGH', 'MEDIUM', 'LOW'), i.incident_id;
+-------------+----------+---------------+-------------+---------+-------+---------------+
| incident_id | priority | status        | owner       | attacks | worst | assets        |
+-------------+----------+---------------+-------------+---------+-------+---------------+
|           4 | CRITICAL | INVESTIGATING | Karan Shah  |       2 |    10 | web-02        |
|           5 | CRITICAL | OPEN          | Unassigned  |       2 |    10 | vpn-01        |
|           3 | HIGH     | CONTAINED     | Rahul Mehta |       3 |    10 | db-01         |
|           7 | HIGH     | INVESTIGATING | Rahul Mehta |       2 |     8 | web-01        |
|           6 | MEDIUM   | OPEN          | Priya Nair  |       5 |     8 | web-01,web-02 |
+-------------+----------+---------------+-------------+---------+-------+---------------+
5 rows in set (0.00 sec)
```

Fig. 68 CQ-02 actual output

CQ-03

Field	Content
ID	CQ-03
Business question	Which assets were attacked through a CVE they still have not patched?

SQL	`SELECT a.hostname, e.event_id, e.cve_id, v.cvss, av.detected_at,`
	`  DATEDIFF('2026-10-28', av.detected_at) AS days_unpatched`
	`FROM attack_event e`
	`JOIN asset_vulnerability av ON av.asset_id = e.asset_id`
	`  AND av.cve_id = e.cve_id`
	`  AND av.patched_at IS NULL`
	`JOIN vulnerability v ON v.cve_id = e.cve_id`
	`JOIN asset a ON a.asset_id = e.asset_id`
	`ORDER BY v.cvss DESC, a.hostname;`

Expected output	5 rows: vpn-01 (CVE-2024-3400), web-02, mail-01, web-01, dns-01. db-01 (MOVEit) and vpn-01 (regreSSHion) must not appear because they were patched before the attack.
Actual output screenshot	Below (Fig. 69)
Explanation	The join to asset_vulnerability uses both asset_id and cve_id plus patched_at IS NULL in the ON clause, so an attack only matches if that exact asset still had that exact CVE open. DATEDIFF gives days since detection.

```
mysql> SELECT a.hostname, e.event_id, e.cve_id, v.cvss, av.detected_at,
    ->        DATEDIFF('2026-10-28', av.detected_at) AS days_unpatched
    -> FROM attack_event e
    -> JOIN asset_vulnerability av ON av.asset_id = e.asset_id
    ->                            AND av.cve_id   = e.cve_id
    ->                            AND av.patched_at IS NULL
    -> JOIN vulnerability v ON v.cve_id   = e.cve_id
    -> JOIN asset a         ON a.asset_id = e.asset_id
    -> ORDER BY v.cvss DESC, a.hostname;
+----------+----------+----------------+------+-------------+----------------+
| hostname | event_id | cve_id         | cvss | detected_at | days_unpatched |
+----------+----------+----------------+------+-------------+----------------+
| vpn-01   |       15 | CVE-2024-3400  | 10.0 | 2026-09-01  |             57 |
| web-02   |       10 | CVE-2022-22965 |  9.8 | 2026-09-05  |             53 |
| mail-01  |        5 | CVE-2023-4966  |  9.4 | 2026-09-12  |             46 |
| web-01   |       14 | CVE-2023-44487 |  7.5 | 2026-09-15  |             43 |
| dns-01   |       28 | CVE-2023-48795 |  5.9 | 2026-09-20  |             38 |
+----------+----------+----------------+------+-------------+----------------+
5 rows in set (0.00 sec)
```

Fig. 69 CQ-03 actual output

CQ-04

Field	Content
ID	CQ-04
Business question	Which malware families cause the most attacks, and who uses them?

SQL	`SELECT m.family,`
	`COUNT(*) AS attacks,`
	`GROUP_CONCAT(DISTINCT COALESCE(t.actor_name, 'Unknown')) AS used_by,`
	`ROUND(100 * COUNT(*) / (SELECT COUNT(*) FROM attack_event WHERE malware_id IS NOT NULL), 1) AS pct_of_malware_attacks`
	`FROM attack_event e`
	`JOIN malware m ON m.malware_id = e.malware_id`
	`LEFT JOIN threat_actor t ON t.actor_id = m.actor_id`
	`GROUP BY m.family`
	`ORDER BY attacks DESC, m.family;`

Expected output	7 families. Botnet (ParrotBot) leads with 4 of the 18 malware attacks, about 22%. Miner is "Unknown" because OrphanMiner has no actor.
Actual output screenshot	Below (Fig. 70)
Explanation	attack_event, malware and threat_actor (LEFT JOIN keeps the actor-less malware). The scalar subquery in the SELECT gives the denominator, the count of attacks that carried any malware.

```
mysql> SELECT m.family,
    ->        COUNT(*) AS attacks,
    ->        GROUP_CONCAT(DISTINCT COALESCE(t.actor_name, 'Unknown')) AS used_by,
    ->        ROUND(100 * COUNT(*) / (SELECT COUNT(*) FROM attack_event
    ->                                WHERE malware_id IS NOT NULL), 1) AS pct_of_malware_attacks
    -> FROM attack_event e
    -> JOIN malware m           ON m.malware_id = e.malware_id
    -> LEFT JOIN threat_actor t ON t.actor_id   = m.actor_id
    -> GROUP BY m.family
    -> ORDER BY attacks DESC, m.family;
+---------+---------+----------------+------------------------+
| family  | attacks | used_by        | pct_of_malware_attacks |
+---------+---------+----------------+------------------------+
| Botnet  |       4 | Ghost Parrot   |                   22.2 |
| Loader  |       3 | Crimson Lynx   |                   16.7 |
| RAT     |       3 | Silent Monsoon |                   16.7 |
| Stealer |       3 | Silent Monsoon |                   16.7 |
| Miner   |       2 | Unknown        |                   11.1 |
| Wiper   |       2 | Iron Kestrel   |                   11.1 |
| Locker  |       1 | Crimson Lynx   |                    5.6 |
+---------+---------+----------------+------------------------+
7 rows in set (0.00 sec)
```

Fig. 70 CQ-04 actual output

CQ-05

Field	Content
ID	CQ-05
Business question	Time machine: how many attacks matched an indicator we already knew about at the time, and how late were we for the rest?

SQL	`SELECT CASE WHEN i.recorded_from <= e.event_time THEN 'known at attack time'`
	`ELSE 'recognised later' END AS knowledge,`
	`COUNT(*) AS matches,`
	`COUNT(DISTINCT e.event_id) AS attacks,`
	`ROUND(AVG(CASE WHEN i.recorded_from > e.event_time`
	`THEN TIMESTAMPDIFF(HOUR, e.event_time, i.recorded_from) / 24`
	`END), 1) AS avg_days_late`
	`FROM attack_event e`
	`JOIN event_ioc ei ON ei.event_id = e.event_id`
	`JOIN ioc i ON i.ioc_version_id = ei.ioc_version_id`
	`GROUP BY knowledge`
	`ORDER BY knowledge;`

Expected output	27 attack-indicator matches: 18 known at attack time (17 attacks) and 9 recognised later (8 attacks), on average about 5 days late.
Actual output screenshot	Below (Fig. 71)
Explanation	Joins attack_event to event_ioc to ioc and compares recorded_from (when we learned) with event_time. The CASE in AVG returns NULL for "known" rows so only late rows are averaged. Hours / 24 keeps fractions of a day.

```
mysql> SELECT CASE WHEN i.recorded_from <= e.event_time THEN 'known at attack time'
    ->             ELSE 'recognised later' END                 AS knowledge,
    ->        COUNT(*)                                          AS matches,
    ->        COUNT(DISTINCT e.event_id)                        AS attacks,
    ->        ROUND(AVG(CASE WHEN i.recorded_from > e.event_time
    ->                       THEN TIMESTAMPDIFF(HOUR, e.event_time, i.recorded_from) / 24
    ->                   END), 1)                              AS avg_days_late
    -> FROM attack_event e
    -> JOIN event_ioc ei ON ei.event_id      = e.event_id
    -> JOIN ioc i        ON i.ioc_version_id = ei.ioc_version_id
    -> GROUP BY knowledge
    -> ORDER BY knowledge;
+----------------------+---------+---------+---------------+
| knowledge            | matches | attacks | avg_days_late |
+----------------------+---------+---------+---------------+
| known at attack time |      18 |      17 |          NULL |
| recognised later     |       9 |       8 |           4.9 |
+----------------------+---------+---------+---------------+
2 rows in set (0.00 sec)
```

Fig. 71 CQ-05 actual output

CQ-06

Field	Content
ID	CQ-06
Business question	What did each retro-hunt find, and what happened to the incident it opened?

SQL	`SELECT r.run_id, i.ioc_value, r.started_at,`
	`COUNT(h.log_id) AS logs_matched,`
	`COUNT(DISTINCT l.dst_asset_id) AS assets_touched,`
	`MIN(l.log_time) AS first_contact,`
	`inc.status AS incident_status`
	`FROM retro_hunt_run r`
	`JOIN ioc i ON i.ioc_version_id = r.ioc_version_id`
	`LEFT JOIN retro_hit h ON h.run_id = r.run_id`
	`LEFT JOIN network_log l ON l.log_id = h.log_id`
	`LEFT JOIN incident inc ON inc.incident_id = r.incident_id`
	`GROUP BY r.run_id, i.ioc_value, r.started_at, inc.status`
	`ORDER BY r.run_id;`

Expected output	2 runs. Run 1 (198.51.100.45) found 4 old log lines on 2 assets, incident INVESTIGATING; run 2 (203.0.113.11) found 4 lines on 2 assets, incident OPEN.
Actual output screenshot	Below (Fig. 72)
Explanation	Five tables. LEFT JOINs to retro_hit and network_log keep runs with zero hits; COUNT(h.log_id) counts hits and COUNT(DISTINCT dst_asset_id) counts assets. This is the same logic as v_retro_hunt_summary.

```
mysql> SELECT r.run_id, i.ioc_value, r.started_at,
    ->        COUNT(h.log_id)               AS logs_matched,
    ->        COUNT(DISTINCT l.dst_asset_id) AS assets_touched,
    ->        MIN(l.log_time)               AS first_contact,
    ->        inc.status                    AS incident_status
    -> FROM retro_hunt_run r
    -> JOIN ioc i           ON i.ioc_version_id = r.ioc_version_id
    -> LEFT JOIN retro_hit h ON h.run_id  = r.run_id
    -> LEFT JOIN network_log l ON l.log_id = h.log_id
    -> LEFT JOIN incident inc  ON inc.incident_id = r.incident_id
    -> GROUP BY r.run_id, i.ioc_value, r.started_at, inc.status
    -> ORDER BY r.run_id;
+--------+--------------+---------------------+--------------+---------------+---------------------+-----------------+
| run_id | ioc_value    | started_at          | logs_matched | assets_touched | first_contact       | incident_status |
+--------+--------------+---------------------+--------------+---------------+---------------------+-----------------+
|      1 | 198.51.100.45 | 2026-10-12 08:00:00 |            4 |             2 | 2026-10-03 03:00:00 | INVESTIGATING   |
|      2 | 203.0.113.11  | 2026-10-20 09:00:00 |            4 |             2 | 2026-10-08 07:00:00 | OPEN            |
+--------+--------------+---------------------+--------------+---------------+---------------------+-----------------+
2 rows in set (0.00 sec)
```

Fig. 72 CQ-06 actual output

CQ-07

Field	Content
ID	CQ-07
Business question	What is the response playbook for the attack type that hits web-01 most often?

SQL	`SELECT t.type_name, m.step_no, m.action`
	`FROM mitigation m`
	`JOIN attack_type t ON t.type_id = m.type_id`
	`WHERE m.type_id = (SELECT e.type_id`
	`FROM attack_event e`
	`JOIN asset a ON a.asset_id = e.asset_id`
	`WHERE a.hostname = 'web-01'`
	`GROUP BY e.type_id`
	`ORDER BY COUNT(*) DESC`
	`LIMIT 1)`
	`ORDER BY m.step_no;`

Expected output	Brute Force (4 of web-01's 7 attacks), with its 3 steps: block the IP, lock the accounts, enforce MFA.
Actual output screenshot	Below (Fig. 73)
Explanation	Nested: the subquery groups web-01's attacks by type, orders by count and keeps the top type; the outer query joins mitigation to attack_type for that type.

```
mysql> SELECT t.type_name, m.step_no, m.action
    -> FROM mitigation m
    -> JOIN attack_type t ON t.type_id = m.type_id
    -> WHERE m.type_id = (SELECT e.type_id
    ->                    FROM attack_event e
    ->                    JOIN asset a ON a.asset_id = e.asset_id
    ->                    WHERE a.hostname = 'web-01'
    ->                    GROUP BY e.type_id
    ->                    ORDER BY COUNT(*) DESC
    ->                    LIMIT 1)
    -> ORDER BY m.step_no;
+-------------+---------+-------------------------------------+
| type_name   | step_no | action                              |
+-------------+---------+-------------------------------------+
| Brute Force |       1 | Block the source IP at the firewall |
| Brute Force |       2 | Lock the targeted accounts          |
| Brute Force |       3 | Enforce MFA on the login page       |
+-------------+---------+-------------------------------------+
3 rows in set (0.00 sec)
```

Fig. 73 CQ-07 actual output

CQ-08

Field	Content
ID	CQ-08
Business question	How is the open-incident workload spread across analysts?

SQL	`SELECT an.full_name,`
	`  COUNT(i.incident_id)  AS open_incidents,`
	`  COALESCE(SUM(i.priority = 'CRITICAL'), 0)  AS critical,`
	`  RANK() OVER (ORDER BY COUNT(i.incident_id) DESC) AS workload_rank`
	`FROM analyst an`
	`LEFT JOIN incident i ON i.assigned_to = an.analyst_id`
	`  AND i.status <> 'CLOSED'`
	`WHERE an.role = 'analyst'`
	`GROUP BY an.analyst_id, an.full_name`
	`ORDER BY workload_rank, an.full_name;`

Expected output	Rahul 2 open incidents (rank 1), Karan and Priya 1 each (rank 2, Karan's is critical), Arjun 0 (rank 4, because RANK skips after the tie).
Actual output screenshot	Below (Fig. 74)
Explanation	Analytic: LEFT JOIN with status <> CLOSED inside the ON clause keeps Arjun with zero. SUM(priority = 'CRITICAL') counts TRUE as 1. RANK() runs after GROUP BY.

```
mysql> SELECT an.full_name,
    ->        COUNT(i.incident_id)                        AS open_incidents,
    ->        COALESCE(SUM(i.priority = 'CRITICAL'), 0)     AS critical,
    ->        RANK() OVER (ORDER BY COUNT(i.incident_id) DESC) AS workload_rank
    -> FROM analyst an
    -> LEFT JOIN incident i ON i.assigned_to = an.analyst_id
    ->                     AND i.status <> 'CLOSED'
    -> WHERE an.role = 'analyst'
    -> GROUP BY an.analyst_id, an.full_name
    -> ORDER BY workload_rank, an.full_name;
+-------------+----------------+----------+---------------+
| full_name   | open_incidents | critical | workload_rank |
+-------------+----------------+----------+---------------+
| Rahul Mehta |              2 |        0 |             1 |
| Karan Shah  |              1 |        1 |             2 |
| Priya Nair  |              1 |        0 |             2 |
| Arjun Das   |              0 |        0 |             4 |
+-------------+----------------+----------+---------------+
4 rows in set (0.00 sec)
```

Fig. 74 CQ-08 actual output

CQ-09

Field	Content
ID	CQ-09
Business question	Which countries do attacks come from, what share does each have, and what is each country's most common attack type?

SQL	`SELECT c.src_country, c.attacks,`
	`  ROUND(100 * c.attacks / SUM(c.attacks) OVER (), 1) AS pct,`
	`  (SELECT t.type_name`
	`  FROM attack_event e2`
	`  JOIN attack_type t ON t.type_id = e2.type_id`
	`  WHERE e2.src_country = c.src_country`
	`  GROUP BY t.type_name`
	`  ORDER BY COUNT(*) DESC, t.type_name`
	`  LIMIT 1) AS top_type`
	`FROM (SELECT src_country, COUNT(*) AS attacks`
	`  FROM attack_event`
	`  GROUP BY src_country) AS c`
	`ORDER BY c.attacks DESC, c.src_country;`

Expected output	7 countries. RU 8 (26.7%), CN 6 (20.0%), BR/KP/US 4 each (13.3%), IN/NL 2 each (6.7%). RU's top type is Brute Force; KP has a 4-way tie, broken alphabetically to Credential Dumping.
Actual output screenshot	Below (Fig. 75)
Explanation	A derived table counts per country; SUM(...) OVER () gives the total for the percentage without a second query. A correlated subquery finds each country's most common type.

```
mysql> SELECT c.src_country, c.attacks,
    ->        ROUND(100 * c.attacks / SUM(c.attacks) OVER (), 1) AS pct,
    ->        (SELECT t.type_name
    ->           FROM attack_event e2
    ->           JOIN attack_type t ON t.type_id = e2.type_id
    ->          WHERE e2.src_country = c.src_country
    ->          GROUP BY t.type_name
    ->          ORDER BY COUNT(*) DESC, t.type_name
    ->          LIMIT 1) AS top_type
    -> FROM (SELECT src_country, COUNT(*) AS attacks
    ->       FROM attack_event
    ->       GROUP BY src_country) AS c
    -> ORDER BY c.attacks DESC, c.src_country;
+-------------+---------+------+---------------------------+
| src_country | attacks | pct  | top_type                  |
+-------------+---------+------+---------------------------+
| RU          |       8 | 26.7 | Brute Force               |
| CN          |       6 | 20.0 | Exploit Public-Facing App |
| BR          |       4 | 13.3 | DDoS                      |
| KP          |       4 | 13.3 | Credential Dumping        |
| US          |       4 | 13.3 | Port Scan                 |
| IN          |       2 |  6.7 | Brute Force               |
| NL          |       2 |  6.7 | DDoS                      |
+-------------+---------+------+---------------------------+
7 rows in set (0.00 sec)
```

Fig. 75 CQ-09 actual output

CQ-10

Field	Content
ID	CQ-10
Business question	Which assets with an unpatched critical CVE (CVSS 9+) have been attacked by an espionage group?

SQL	`SELECT DISTINCT a.hostname, av.cve_id, v.cvss, t.actor_name`
	`FROM asset_vulnerability av`
	`JOIN vulnerability v  ON v.cve_id   = av.cve_id`
	`JOIN asset a          ON a.asset_id = av.asset_id`
	`JOIN attack_event e   ON e.asset_id = av.asset_id`
	`JOIN threat_actor t   ON t.actor_id = e.actor_id`
	`WHERE av.patched_at IS NULL`
	`  AND v.cvss >= 9`
	`  AND t.motive = 'espionage'`
	`ORDER BY v.cvss DESC, a.hostname;`

Expected output	2 rows: vpn-01 with CVE-2024-3400 (10.0) and mail-01 with CVE-2023-4966 (9.4), both attacked by Silent Monsoon.
Actual output screenshot	Below (Fig. 76)
Explanation	Five tables. Starts from unpatched exposures with CVSS 9+, then joins attacks on the same asset by espionage groups. DISTINCT removes duplicates when a group attacked the asset more than once.

```
mysql> SELECT DISTINCT a.hostname, av.cve_id, v.cvss, t.actor_name
    -> FROM asset_vulnerability av
    -> JOIN vulnerability v  ON v.cve_id   = av.cve_id
    -> JOIN asset a          ON a.asset_id = av.asset_id
    -> JOIN attack_event e   ON e.asset_id = av.asset_id
    -> JOIN threat_actor t   ON t.actor_id = e.actor_id
    -> WHERE av.patched_at IS NULL
    ->   AND v.cvss >= 9
    ->   AND t.motive = 'espionage'
    -> ORDER BY v.cvss DESC, a.hostname;
+----------+---------------+------+----------------+
| hostname | cve_id        | cvss | actor_name     |
+----------+---------------+------+----------------+
| vpn-01   | CVE-2024-3400 | 10.0 | Silent Monsoon |
| mail-01  | CVE-2023-4966 |  9.4 | Silent Monsoon |
+----------+---------------+------+----------------+
2 rows in set (0.00 sec)
```

Fig. 76 CQ-10 actual output

CQ-11

Field	Content
ID	CQ-11
Business question	Which source IPs launched a follow-up attack within one hour (attack bursts)?

SQL	`SELECT src_ip, prev_event, event_id, gap_minutes, hostname`
	`FROM (SELECT e.src_ip, e.event_id, a.hostname,`
	`LAG(e.event_id) OVER w AS prev_event,`
	`TIMESTAMPDIFF(MINUTE, LAG(e.event_time) OVER w, e.event_time)`
	`AS gap_minutes`
	`FROM attack_event e`
	`JOIN asset a ON a.asset_id = e.asset_id`
	`WINDOW w AS (PARTITION BY e.src_ip ORDER BY e.event_time)) AS x`
	`WHERE gap_minutes <= 60`
	`ORDER BY gap_minutes, src_ip;`

Expected output	7 bursts. The fastest is 2 minutes (203.0.113.200 scanning two hosts), the slowest within the hour is 45 minutes (192.0.2.50 on db-01).
Actual output screenshot	Below (Fig. 77)
Explanation	LAG over a window partitioned by source IP gives each attack's previous attack from the same IP; the outer query keeps gaps of 60 minutes or less. The filter needs a derived table because window results cannot be used in WHERE.

```
mysql> SELECT src_ip, prev_event, event_id, gap_minutes, hostname
    -> FROM (SELECT e.src_ip, e.event_id, a.hostname,
    ->              LAG(e.event_id) OVER w AS prev_event,
    ->              TIMESTAMPDIFF(MINUTE, LAG(e.event_time) OVER w, e.event_time) AS gap_minutes
    ->       FROM attack_event e
    ->       JOIN asset a ON a.asset_id = e.asset_id
    ->       WINDOW w AS (PARTITION BY e.src_ip ORDER BY e.event_time)) AS x
    -> WHERE gap_minutes <= 60
    -> ORDER BY gap_minutes, src_ip;
+---------------+------------+----------+-------------+----------+
| src_ip        | prev_event | event_id | gap_minutes | hostname |
+---------------+------------+----------+-------------+----------+
| 203.0.113.200 |         12 |       13 |           2 | vpn-01   |
| 198.51.100.45 |         23 |       24 |           3 | web-01   |
| 198.51.100.88 |          8 |        9 |           5 | web-02   |
| 198.51.100.23 |          1 |        2 |           6 | web-01   |
| 203.0.113.11  |         15 |       16 |          25 | vpn-01   |
| 198.51.100.45 |         10 |       11 |          37 | web-02   |
| 192.0.2.50    |          6 |        7 |          45 | db-01    |
+---------------+------------+----------+-------------+----------+
7 rows in set (0.00 sec)
```

Fig. 77 CQ-11 actual output

12.2 Views

Three new views, plus the two Review 1 views (v_vulnerability for CVE severity and v_retro_hunt_summary for retro-hunt counts) that replaced stored columns.

v_attack_details (joins 5 tables)

Field	Content
Purpose	One flat row per attack with host, type, actor and malware names, so the dashboard and search pages need no joins.

Field	Content
CREATE VIEW statement, query using the view and screenshot	Fig. 78 below
Why useful	Abstraction and simplification: pages query one object. **Not updatable:** information_schema reports IS_UPDATABLE = NO because it uses outer joins, and the DELETE in the screenshot is refused (ERROR 1288). We only grant SELECT on it.

```
mysql> CREATE VIEW v_attack_details AS
    -> SELECT e.event_id, e.event_time, e.src_ip, e.src_country,
    ->        a.hostname, t.type_name,
    ->        ta.actor_name, m.malware_name, e.cve_id,
    ->        e.severity, e.blocked
    -> FROM attack_event e
    -> JOIN asset a             ON a.asset_id   = e.asset_id
    -> JOIN attack_type t       ON t.type_id    = e.type_id
    -> LEFT JOIN threat_actor ta ON ta.actor_id  = e.actor_id
    -> LEFT JOIN malware m       ON m.malware_id = e.malware_id;
Query OK, 0 rows affected (0.02 sec)

mysql> SELECT event_id, event_time, src_ip, type_name, actor_name, severity
    -> FROM v_attack_details
    -> WHERE hostname = 'web-01'
    -> ORDER BY event_time;
+----------+---------------------+--------------+-------------+-------------+----------+
| event_id | event_time          | src_ip       | type_name   | actor_name  | severity |
+----------+---------------------+--------------+-------------+-------------+----------+
|        1 | 2026-10-01 02:14:00 | 198.51.100.23 | Brute Force | Crimson Lynx |        6 |
|        2 | 2026-10-01 02:20:00 | 198.51.100.23 | Brute Force | Crimson Lynx |        7 |
|        3 | 2026-10-02 11:05:00 | 203.0.113.200 | Port Scan   | NULL         |        3 |
|       14 | 2026-10-09 16:45:00 | 192.0.2.77    | DDoS        | Byte Bandits |        8 |
|       19 | 2026-10-13 07:05:00 | 198.51.100.88 | DDoS        | Ghost Parrot |        6 |
|       23 | 2026-10-18 02:02:00 | 198.51.100.45 | Brute Force | Crimson Lynx |        7 |
|       24 | 2026-10-18 02:05:00 | 198.51.100.45 | Brute Force | Crimson Lynx |        8 |
+----------+---------------------+--------------+-------------+-------------+----------+
7 rows in set (0.00 sec)

mysql> DELETE FROM v_attack_details WHERE event_id = 3;
ERROR 1288 (HY000): The target table v_attack_details of the DELETE is not updatable
```

Fig. 78 v_attack_details: CREATE VIEW, query using it, and update test

v_asset_risk (aggregated)

Field	Content
Purpose	Per asset: attacks, attacks not blocked, worst severity and open vulnerabilities, for the risk panel.
CREATE VIEW statement, query using the view and screenshot	Fig. 79 below
Why useful	Simplification: the aggregation is written once. **Not updatable** because of GROUP BY and COUNT/SUM; MySQL refuses the UPDATE (ERROR 1288). The viewer role can read it without any access to the base tables.

```
mysql> CREATE VIEW v_asset_risk AS
    -> SELECT a.asset_id, a.hostname, a.criticality,
    ->        COUNT(e.event_id)              AS attacks,
    ->        COALESCE(SUM(e.blocked = 0), 0) AS not_blocked,
    ->        MAX(e.severity)                AS worst_severity,
    ->        (SELECT COUNT(*) FROM asset_vulnerability av
    ->          WHERE av.asset_id = a.asset_id AND av.patched_at IS NULL) AS open_vulns
    -> FROM asset a
    -> LEFT JOIN attack_event e ON e.asset_id = a.asset_id
    -> GROUP BY a.asset_id, a.hostname, a.criticality;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT hostname, criticality, attacks, not_blocked, worst_severity, open_vulns
    -> FROM v_asset_risk
    -> ORDER BY not_blocked DESC, criticality DESC
    -> LIMIT 5;
+----------+-------------+---------+-------------+----------------+------------+
| hostname | criticality | attacks | not_blocked | worst_severity | open_vulns |
+----------+-------------+---------+-------------+----------------+------------+
| web-01   |           5 |       7 |           4 |              8 |          1 |
| db-01    |           5 |       4 |           2 |             10 |          0 |
| web-02   |           4 |       5 |           2 |             10 |          1 |
| vpn-01   |           4 |       4 |           2 |             10 |          1 |
| mail-01  |           3 |       3 |           2 |              9 |          1 |
+----------+-------------+---------+-------------+----------------+------------+
5 rows in set (0.00 sec)

mysql> UPDATE v_asset_risk SET criticality = 5 WHERE asset_id = 2;
ERROR 1288 (HY000): The target table v_asset_risk of the UPDATE is not updatable
```

Fig. 79 v_asset_risk: CREATE VIEW, query using it, and update test

v_my_incidents (security, row-restricting)

Field	Content
Purpose	Shows each analyst only the incidents assigned to them, by matching the MySQL login (USER()) to the analyst's email.
CREATE VIEW statement, query using the view and screenshot	Fig. 80 below
Why useful	Security: analysts have no privilege on the incident table, only on this view. **Updatable** (IS_UPDATABLE = YES): it reads one table with no aggregation or join. WITH CHECK OPTION blocks changes that would move a row out of the view, such as reassigning an incident to someone else (section 12.7). As root it shows 0 rows, because root is not an analyst.

```
mysql> CREATE VIEW v_my_incidents AS
    -> SELECT incident_id, title, priority, status, assigned_to, opened_at, closed_at
    -> FROM incident
    -> WHERE assigned_to = (SELECT analyst_id FROM analyst
    ->                      WHERE email = CONCAT(SUBSTRING_INDEX(USER(), '@', 1), '@sentinelx.in'))
    -> WITH CHECK OPTION;
Query OK, 0 rows affected (0.13 sec)

mysql> SELECT USER() AS logged_in_as, COUNT(*) AS rows_visible FROM v_my_incidents;
+----------------+--------------+
| logged_in_as   | rows_visible |
+----------------+--------------+
| root@localhost |            0 |
+----------------+--------------+
1 row in set (0.00 sec)

mysql> SELECT table_name, is_updatable
    -> FROM information_schema.views
    -> WHERE table_schema = 'sentinelx'
    -> ORDER BY table_name;
+------------------+--------------+
| TABLE_NAME       | IS_UPDATABLE |
+------------------+--------------+
| v_asset_risk     | NO           |
| v_attack_details | NO           |
| v_my_incidents   | YES          |
+------------------+--------------+
3 rows in set (0.01 sec)
```

Fig. 80 v_my_incidents: CREATE VIEW, query using it, and update test

Review 1 views

```
mysql> CREATE VIEW v_vulnerability AS
    -> SELECT cve_id, cvss, published,
    ->        CASE WHEN cvss >= 9 THEN 'CRITICAL'
    ->             WHEN cvss >= 7 THEN 'HIGH'
    ->             WHEN cvss >= 4 THEN 'MEDIUM'
    ->             ELSE 'LOW' END AS severity
    -> FROM vulnerability;
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE VIEW v_retro_hunt_summary AS
    -> SELECT r.run_id, r.ioc_version_id, r.started_at, r.incident_id,
    ->        COUNT(h.log_id)                AS logs_matched,
    ->        COUNT(DISTINCT l.dst_asset_id) AS assets_touched
    -> FROM retro_hunt_run r
    -> LEFT JOIN retro_hit   h ON h.run_id = r.run_id
    -> LEFT JOIN network_log l ON l.log_id = h.log_id
    -> GROUP BY r.run_id, r.ioc_version_id, r.started_at, r.incident_id;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM v_retro_hunt_summary;
+--------+----------------+---------------------+-------------+--------------+----------------+
| run_id | ioc_version_id | started_at          | incident_id | logs_matched | assets_touched |
+--------+----------------+---------------------+-------------+--------------+----------------+
|      1 |              2 | 2026-10-12 08:00:00 |           4 |            4 |              2 |
|      2 |              4 | 2026-10-20 09:00:00 |           5 |            4 |              2 |
+--------+----------------+---------------------+-------------+--------------+----------------+
2 rows in set (0.00 sec)
```

Fig. 81 v_vulnerability and v_retro_hunt_summary

12.3 Stored Procedures

sp_assign_incident(incident, analyst) assigns an incident inside a transaction. It locks the incident row with SELECT ... FOR UPDATE, so two analysts cannot claim it at the same moment, and rejects closed incidents and viewer accounts with SIGNAL. Any error rolls the transaction back and is passed to the caller (RESIGNAL).

```
mysql> CREATE PROCEDURE sp_assign_incident(IN p_incident INT, IN p_analyst INT)
    -> BEGIN
    ->   DECLARE v_status VARCHAR(15);
    ->   DECLARE v_role   VARCHAR(10);
    ->   DECLARE EXIT HANDLER FOR SQLEXCEPTION
    ->   BEGIN
    ->     ROLLBACK;
    ->     RESIGNAL;
    ->   END;
    ->
    ->   START TRANSACTION;
    ->   SELECT status INTO v_status FROM incident
    ->    WHERE incident_id = p_incident FOR UPDATE;
    ->   IF v_status IS NULL THEN
    ->     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'No such incident';
    ->   ELSEIF v_status = 'CLOSED' THEN
    ->     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Incident is already closed';
    ->   END IF;
    ->
    ->   SELECT role INTO v_role FROM analyst WHERE analyst_id = p_analyst;
    ->   IF v_role IS NULL OR v_role = 'viewer' THEN
    ->     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Only analysts or admins can own incidents';
    ->   END IF;
    ->
    ->   UPDATE incident
    ->      SET assigned_to = p_analyst,
    ->          status = IF(status = 'OPEN', 'INVESTIGATING', status)
    ->    WHERE incident_id = p_incident;
    ->   COMMIT;
    ->
    ->   SELECT incident_id, status, assigned_to FROM incident WHERE incident_id = p_incident;
    -> END;
Query OK, 0 rows affected (0.05 sec)
```

Fig. 82 sp_assign_incident: definition, then one successful and two rejected calls (1 of 2)

```
mysql> CALL sp_assign_incident(5, 6);
+-------------+---------------+-------------+
| incident_id | status        | assigned_to |
+-------------+---------------+-------------+
|           5 | INVESTIGATING |           6 |
+-------------+---------------+-------------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

mysql> CALL sp_assign_incident(1, 4);
ERROR 1644 (45000): Incident is already closed

mysql> CALL sp_assign_incident(6, 5);
ERROR 1644 (45000): Only analysts or admins can own incidents
```

Fig. 83 sp_assign_incident: definition, then one successful and two rejected calls (2 of 2)

sp_attack_report(from, to) returns the attack summary by type for a date range. The viewer role gets EXECUTE on it instead of access to the attack table.

```
mysql> CREATE PROCEDURE sp_attack_report(IN p_from DATE, IN p_to DATE)
    -> READS SQL DATA
    -> BEGIN
    ->   SELECT t.type_name,
    ->          COUNT(*)        AS attacks,
    ->          SUM(e.blocked)  AS blocked,
    ->          MAX(e.severity) AS worst
    ->   FROM attack_event e
    ->   JOIN attack_type t ON t.type_id = e.type_id
    ->   WHERE e.event_time >= p_from
    ->     AND e.event_time <  p_to + INTERVAL 1 DAY
    ->   GROUP BY t.type_name
    ->   ORDER BY attacks DESC, t.type_name;
    -> END;
Query OK, 0 rows affected (0.05 sec)
```

Fig. 84 sp_attack_report: definition and call (1 of 2)

```
mysql> CALL sp_attack_report('2026-10-01', '2026-10-15');
+---------------------------+---------+---------+-------+
| type_name                 | attacks | blocked | worst |
+---------------------------+---------+---------+-------+
| Exploit Public-Facing App |       5 |       2 |    10 |
| DDoS                      |       4 |       3 |     8 |
| Brute Force               |       3 |       1 |     7 |
| Phishing                  |       3 |       2 |     6 |
| Port Scan                 |       3 |       3 |     3 |
| Credential Dumping        |       2 |       0 |     9 |
| Ransomware                |       1 |       0 |    10 |
+---------------------------+---------+---------+-------+
7 rows in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

Fig. 85 sp_attack_report: definition and call (2 of 2)

12.4 User-Defined Functions

fn_severity_label(cvss) is DETERMINISTIC and turns a CVSS score into LOW, MEDIUM, HIGH or CRITICAL. **fn_asset_risk(asset_id)** reads three tables and returns criticality x 10 + open vulnerabilities x 5 + attacks not blocked x 2.

```
mysql> CREATE FUNCTION fn_severity_label(p_cvss DECIMAL(3,1))
    -> RETURNS VARCHAR(8)
    -> DETERMINISTIC NO SQL
    -> BEGIN
    ->   RETURN CASE WHEN p_cvss >= 9 THEN 'CRITICAL'
    ->               WHEN p_cvss >= 7 THEN 'HIGH'
    ->               WHEN p_cvss >= 4 THEN 'MEDIUM'
    ->               ELSE 'LOW' END;
    -> END;
Query OK, 0 rows affected (0.03 sec)
```

Fig. 86 fn_severity_label: definition and use (1 of 2)

```
mysql> SELECT cve_id, cvss, fn_severity_label(cvss) AS label
    -> FROM vulnerability
    -> ORDER BY cvss DESC, cve_id;
+----------------+------+----------+
| cve_id         | cvss | label    |
+----------------+------+----------+
| CVE-2021-44228 | 10.0 | CRITICAL |
| CVE-2024-3400  | 10.0 | CRITICAL |
| CVE-2019-0708  |  9.8 | CRITICAL |
| CVE-2022-22965 |  9.8 | CRITICAL |
| CVE-2023-34362 |  9.8 | CRITICAL |
| CVE-2023-4966  |  9.4 | CRITICAL |
| CVE-2024-6387  |  8.1 | HIGH     |
| CVE-2023-44487 |  7.5 | HIGH     |
| CVE-2023-48795 |  5.9 | MEDIUM   |
+----------------+------+----------+
9 rows in set (0.00 sec)
```

Fig. 87 fn_severity_label: definition and use (2 of 2)

```
mysql> CREATE FUNCTION fn_asset_risk(p_asset INT)
    -> RETURNS INT
    -> READS SQL DATA
    -> BEGIN
    ->   DECLARE v_crit, v_open, v_hits INT DEFAULT 0;
    ->   SELECT criticality INTO v_crit FROM asset WHERE asset_id = p_asset;
    ->   SELECT COUNT(*) INTO v_open FROM asset_vulnerability
    ->    WHERE asset_id = p_asset AND patched_at IS NULL;
    ->   SELECT COUNT(*) INTO v_hits FROM attack_event
    ->    WHERE asset_id = p_asset AND blocked = 0;
    ->   RETURN v_crit * 10 + v_open * 5 + v_hits * 2;
    -> END;
Query OK, 0 rows affected (0.01 sec)
```

Fig. 88 fn_asset_risk: definition and top 5 assets by risk (1 of 2)

```
mysql> SELECT hostname, criticality, fn_asset_risk(asset_id) AS risk_score
    -> FROM asset
    -> ORDER BY risk_score DESC, hostname
    -> LIMIT 5;
+----------+-------------+------------+
| hostname | criticality | risk_score |
+----------+-------------+------------+
| web-01   |           5 |         63 |
| db-01    |           5 |         54 |
| vpn-01   |           4 |         49 |
| web-02   |           4 |         49 |
| dns-01   |           4 |         47 |
+----------+-------------+------------+
5 rows in set (0.00 sec)
```

Fig. 89 fn_asset_risk: definition and top 5 assets by risk (2 of 2)

12.5 Triggers

trg_attack_auto_incident (AFTER INSERT on attack_event) opens an incident for any attack of severity 8 or more. If the same asset already has an open incident for the same attack type from the last hour, the attack joins it instead. In the demo three attacks are inserted: the first (severity 9) opens incident 8, the second (severity 8, twenty minutes later) is added to it, and the third (severity 3) is ignored.

```
mysql> CREATE TRIGGER trg_attack_auto_incident
    -> AFTER INSERT ON attack_event
    -> FOR EACH ROW
    -> BEGIN
    ->   DECLARE v_inc INT;
    ->   IF NEW.severity >= 8 THEN
    ->
    ->     SET v_inc = (SELECT ie.incident_id
    ->                  FROM incident_event ie
    ->                  JOIN attack_event e ON e.event_id    = ie.event_id
    ->                  JOIN incident i     ON i.incident_id = ie.incident_id
    ->                  WHERE e.asset_id = NEW.asset_id
    ->                    AND e.type_id  = NEW.type_id
    ->                    AND i.status  <> 'CLOSED'
    ->                    AND e.event_time >= NEW.event_time - INTERVAL 1 HOUR
    ->                  ORDER BY i.opened_at DESC
    ->                  LIMIT 1);
    ->     IF v_inc IS NULL THEN
    ->       INSERT INTO incident (title, priority, status, opened_at)
    ->       VALUES (CONCAT((SELECT type_name FROM attack_type WHERE type_id = NEW.type_id),
    ->                      ' on ', (SELECT hostname FROM asset WHERE asset_id = NEW.asset_id),
    ->                      ' from ', NEW.src_ip),
    ->               IF(NEW.severity >= 9, 'CRITICAL', 'HIGH'), 'OPEN', NEW.event_time);
    ->       SET v_inc = LAST_INSERT_ID();
    ->     END IF;
    ->     INSERT INTO incident_event (incident_id, event_id) VALUES (v_inc, NEW.event_id);
    ->   END IF;
    -> END;
Query OK, 0 rows affected (0.09 sec)
```

Fig. 90 trg_attack_auto_incident: definition and demonstration (1 of 2)

```
mysql> SELECT COUNT(*) AS incidents_before FROM incident;
+------------------+
| incidents_before |
+------------------+
|                7 |
+------------------+
1 row in set (0.00 sec)

mysql> INSERT INTO attack_event (event_time, src_ip, src_country, asset_id, type_id, severity, blocked)
    -> VALUES ('2026-10-27 22:00:00', '203.0.113.66', 'US', 3, 3, 9, 0);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO attack_event (event_time, src_ip, src_country, asset_id, type_id, severity, blocked)
    -> VALUES ('2026-10-27 22:20:00', '203.0.113.66', 'US', 3, 3, 8, 0);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO attack_event (event_time, src_ip, src_country, asset_id, type_id, severity, blocked)
    -> VALUES ('2026-10-27 22:30:00', '203.0.113.66', 'US', 3, 4, 3, 1);
Query OK, 1 row affected (0.00 sec)

mysql> SELECT i.incident_id, i.title, i.priority, i.status, COUNT(ie.event_id) AS attacks
    -> FROM incident i
    -> JOIN incident_event ie ON ie.incident_id = i.incident_id
    -> WHERE i.incident_id > 7
    -> GROUP BY i.incident_id, i.title, i.priority, i.status;
+-------------+-----------------------------------------------------+----------+--------+---------+
| incident_id | title                                               | priority | status | attacks |
+-------------+-----------------------------------------------------+----------+--------+---------+
|           8 | Exploit Public-Facing App on db-01 from 203.0.113.66 | CRITICAL | OPEN   |       2 |
+-------------+-----------------------------------------------------+----------+--------+---------+
1 row in set (0.00 sec)
```

Fig. 91 trg_attack_auto_incident: definition and demonstration (2 of 2)

Three triggers on ioc: **trg_ioc_no_overlap** (BEFORE INSERT) rejects a new version whose valid period overlaps an existing one, **trg_ioc_no_delete** (BEFORE DELETE) keeps the history append-only, and **trg_ioc_retro_hunt** (AFTER INSERT) runs the retro-hunt.

```
mysql> CREATE TRIGGER trg_ioc_no_overlap
    -> BEFORE INSERT ON ioc
    -> FOR EACH ROW
    -> BEGIN
    ->   IF EXISTS (SELECT 1 FROM ioc
    ->              WHERE ioc_type = NEW.ioc_type
    ->                AND ioc_value = NEW.ioc_value
    ->                AND (valid_to IS NULL OR valid_to > NEW.valid_from)
    ->                AND (NEW.valid_to IS NULL OR NEW.valid_to > valid_from)) THEN
    ->     SIGNAL SQLSTATE '45000'
    ->       SET MESSAGE_TEXT = 'An overlapping version of this indicator already exists';
    ->   END IF;
    -> END;
Query OK, 0 rows affected (0.05 sec)
```

Fig. 92 The three triggers on ioc (1 of 3)

```
mysql> CREATE TRIGGER trg_ioc_no_delete
    -> BEFORE DELETE ON ioc
    -> FOR EACH ROW
    -> BEGIN
    ->   SIGNAL SQLSTATE '45000'
    ->     SET MESSAGE_TEXT = 'Indicator history is append-only; close it with recorded_to instead';
    -> END;
Query OK, 0 rows affected (0.02 sec)
```

Fig. 93 The three triggers on ioc (2 of 3)

```
mysql> CREATE TRIGGER trg_ioc_retro_hunt
    -> AFTER INSERT ON ioc
    -> FOR EACH ROW
    -> BEGIN
    ->   DECLARE v_run, v_hits, v_assets INT;
    ->   IF NEW.ioc_type = 'ip' THEN
    ->     INSERT INTO retro_hunt_run (ioc_version_id, started_at)
    ->     VALUES (NEW.ioc_version_id, NEW.recorded_from);
    ->     SET v_run = LAST_INSERT_ID();
    ->
    ->
    ->     INSERT INTO retro_hit (run_id, log_id)
    ->     SELECT v_run, log_id
    ->     FROM network_log
    ->     WHERE src_ip = NEW.ioc_value
    ->       AND log_time >= NEW.valid_from
    ->       AND log_time <  NEW.recorded_from
    ->       AND (NEW.valid_to IS NULL OR log_time < NEW.valid_to);
    ->     SET v_hits = ROW_COUNT();
    ->
    ->     IF v_hits > 0 THEN
    ->       SET v_assets = (SELECT COUNT(DISTINCT l.dst_asset_id)
    ->                       FROM retro_hit h JOIN network_log l ON l.log_id = h.log_id
    ->                       WHERE h.run_id = v_run);
    ->       INSERT INTO incident (title, priority, status, opened_at)
    ->       VALUES (CONCAT('Retro-hunt: ', NEW.ioc_value, ' hit ', v_assets,
    ->                      ' assets ', v_hits, ' times'),
    ->               IF(v_assets >= 3, 'CRITICAL', 'HIGH'), 'OPEN', NEW.recorded_from);
    ->       UPDATE retro_hunt_run SET incident_id = LAST_INSERT_ID() WHERE run_id = v_run;
    ->     END IF;
    ->   END IF;
    -> END;
Query OK, 0 rows affected (0.01 sec)
```

Fig. 94 The three triggers on ioc (3 of 3)

In the demonstration, 198.51.100.77 is added as a bad IP that was hostile from 15 September. The trigger finds the 8 old log lines it sent to 3 assets before anyone knew, and opens a CRITICAL incident. The overlapping second version and the DELETE are both refused.

```
mysql> INSERT INTO ioc (ioc_type, ioc_value, valid_from, recorded_from)
    -> VALUES ('ip', '198.51.100.77', '2026-09-15 00:00:00', '2026-10-28 10:00:00');
Query OK, 1 row affected (0.02 sec)

mysql> SELECT * FROM v_retro_hunt_summary WHERE ioc_version_id = LAST_INSERT_ID();
+--------+----------------+---------------------+-------------+--------------+----------------+
| run_id | ioc_version_id | started_at          | incident_id | logs_matched | assets_touched |
+--------+----------------+---------------------+-------------+--------------+----------------+
|      3 |             13 | 2026-10-28 10:00:00 |           9 |            8 |              3 |
+--------+----------------+---------------------+-------------+--------------+----------------+
1 row in set (0.00 sec)

mysql> SELECT incident_id, title, priority, status, opened_at
    -> FROM incident
    -> WHERE title LIKE 'Retro-hunt: 198.51.100.77%';
+-------------+-----------------------------------------------+----------+--------+---------------------+
| incident_id | title                                         | priority | status | opened_at           |
+-------------+-----------------------------------------------+----------+--------+---------------------+
|           9 | Retro-hunt: 198.51.100.77 hit 3 assets 8 times | CRITICAL | OPEN   | 2026-10-28 10:00:00 |
+-------------+-----------------------------------------------+----------+--------+---------------------+
1 row in set (0.00 sec)

mysql> INSERT INTO ioc (ioc_type, ioc_value, valid_from, recorded_from)
    -> VALUES ('ip', '198.51.100.77', '2026-10-01 00:00:00', '2026-10-28 11:00:00');
ERROR 1644 (45000): An overlapping version of this indicator already exists

mysql> DELETE FROM ioc WHERE ioc_value = '198.51.100.77';
ERROR 1644 (45000): Indicator history is append-only; close it with recorded_to instead
```

Fig. 95 Retro-hunt trigger in action

12.6 Cursors

sp_escalate_incidents walks through every incident that is not closed with a cursor, and raises its priority when its worst attack demands it: to CRITICAL for severity 10, and from LOW or MEDIUM to HIGH for severity 8. It escalated incident 3 (HIGH to CRITICAL, because of attack 30) and incident 6 (MEDIUM to HIGH).

```
mysql> CREATE PROCEDURE sp_escalate_incidents()
    -> BEGIN
    ->   DECLARE done INT DEFAULT 0;
    ->   DECLARE v_id INT;
    ->   DECLARE v_pri VARCHAR(8);
    ->   DECLARE v_max TINYINT;
    ->   DECLARE cur CURSOR FOR
    ->     SELECT i.incident_id, i.priority, MAX(e.severity)
    ->     FROM incident i
    ->     JOIN incident_event ie ON ie.incident_id = i.incident_id
    ->     JOIN attack_event e    ON e.event_id     = ie.event_id
    ->     WHERE i.status <> 'CLOSED'
    ->     GROUP BY i.incident_id, i.priority
    ->     ORDER BY i.incident_id;
    ->   DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
    ->
    ->   DROP TEMPORARY TABLE IF EXISTS escalation_log;
    ->   CREATE TEMPORARY TABLE escalation_log (
    ->     incident_id INT, old_priority VARCHAR(8), new_priority VARCHAR(8), worst_severity TINYINT);
    ->
    ->   OPEN cur;
    ->   read_loop: LOOP
    ->     FETCH cur INTO v_id, v_pri, v_max;
    ->     IF done THEN LEAVE read_loop; END IF;
    ->     IF v_max >= 10 AND v_pri <> 'CRITICAL' THEN
    ->       UPDATE incident SET priority = 'CRITICAL' WHERE incident_id = v_id;
    ->       INSERT INTO escalation_log VALUES (v_id, v_pri, 'CRITICAL', v_max);
    ->     ELSEIF v_max >= 8 AND v_pri IN ('LOW', 'MEDIUM') THEN
    ->       UPDATE incident SET priority = 'HIGH' WHERE incident_id = v_id;
    ->       INSERT INTO escalation_log VALUES (v_id, v_pri, 'HIGH', v_max);
    ->     END IF;
    ->   END LOOP;
    ->   CLOSE cur;
    ->
    ->   SELECT * FROM escalation_log;
    -> END;
Query OK, 0 rows affected (0.05 sec)
```

Fig. 96 Cursor 1: sp_escalate_incidents (1 of 2)

```
mysql> CALL sp_escalate_incidents();
+-------------+--------------+--------------+----------------+
| incident_id | old_priority | new_priority | worst_severity |
+-------------+--------------+--------------+----------------+
|           3 | HIGH         | CRITICAL     |             10 |
|           6 | MEDIUM       | HIGH         |              8 |
+-------------+--------------+--------------+----------------+
2 rows in set (0.01 sec)

Query OK, 0 rows affected (0.01 sec)
```

Fig. 97 Cursor 1: sp_escalate_incidents (2 of 2)

sp_asset_risk_report loops over all assets with a cursor, calls fn_asset_risk for each, puts the score into a temporary table with a band (HIGH 60+, MEDIUM 30+, LOW) and returns it sorted.

```
mysql> CREATE PROCEDURE sp_asset_risk_report()
    -> BEGIN
    ->   DECLARE done INT DEFAULT 0;
    ->   DECLARE v_id INT;
    ->   DECLARE v_host VARCHAR(63);
    ->   DECLARE v_score INT;
    ->   DECLARE cur CURSOR FOR SELECT asset_id, hostname FROM asset ORDER BY asset_id;
    ->   DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
    ->
    ->   DROP TEMPORARY TABLE IF EXISTS risk_report;
    ->   CREATE TEMPORARY TABLE risk_report (hostname VARCHAR(63), risk_score INT, band VARCHAR(8));
    ->
    ->   OPEN cur;
    ->   read_loop: LOOP
    ->     FETCH cur INTO v_id, v_host;
    ->     IF done THEN LEAVE read_loop; END IF;
    ->     SET v_score = fn_asset_risk(v_id);
    ->     INSERT INTO risk_report VALUES (v_host, v_score,
    ->       CASE WHEN v_score >= 60 THEN 'HIGH' WHEN v_score >= 30 THEN 'MEDIUM' ELSE 'LOW' END);
    ->   END LOOP;
    ->   CLOSE cur;
    ->
    ->   SELECT * FROM risk_report ORDER BY risk_score DESC, hostname;
    -> END;
Query OK, 0 rows affected (0.02 sec)
```

Fig. 98 Cursor 2: sp_asset_risk_report (1 of 2)

```
mysql> CALL sp_asset_risk_report();
+-----------+------------+--------+
| hostname  | risk_score | band   |
+-----------+------------+--------+
| web-01    |         63 | HIGH   |
| db-01     |         58 | MEDIUM |
| vpn-01    |         49 | MEDIUM |
| web-02    |         49 | MEDIUM |
| dns-01    |         47 | MEDIUM |
| mail-01   |         39 | MEDIUM |
| ws-fin-01 |         30 | MEDIUM |
| ws-hr-01  |         25 | LOW    |
| ws-dev-01 |         20 | LOW    |
| cam-01    |         12 | LOW    |
+-----------+------------+--------+
10 rows in set (0.01 sec)

Query OK, 0 rows affected (0.01 sec)
```

Fig. 99 Cursor 2: sp_asset_risk_report (2 of 2)

12.7 SQL Security Implementation

Three roles carry the privileges and each user gets one role by default. Restricted roles reach data only through views and procedures, which run with their definer's rights.

Object	sx_admin	sx_analyst	sx_viewer
All tables	ALL	SELECT on 10 tables (not incident, analyst, network_log or retro tables)	-
ioc	ALL	SELECT, INSERT	-
v_my_incidents	ALL	SELECT, UPDATE (own rows only)	-
v_attack_details	ALL	SELECT	-
v_asset_risk	ALL	-	SELECT
sp_assign_incident	ALL	EXECUTE	-
sp_attack_report	ALL	-	EXECUTE
User (default role)	ananya.rao	rahul.mehta	divya.iyer

Task	Evidence
Create 3 or more users/roles	CREATE ROLE and CREATE USER script with default roles (Fig. 100)
GRANT privileges per role matrix	GRANT script and SHOW GRANTS output (Fig. 100, 101)
REVOKE demonstration	SHOW GRANTS before and after REVOKE (Fig. 102)
Least privilege proof	Viewer and analyst log in and try forbidden operations: ERROR 1142 and ERROR 1369 (Fig. 103, 104)
Secure exposure	Viewer reads v_asset_risk and calls sp_attack_report but cannot read attack_event; analyst works on incidents only through v_my_incidents (Fig. 103, 104)

```
mysql> CREATE ROLE 'sx_admin', 'sx_analyst', 'sx_viewer';
Query OK, 0 rows affected (0.01 sec)

mysql> GRANT ALL PRIVILEGES ON sentinelx.* TO 'sx_admin';
Query OK, 0 rows affected (0.01 sec)

mysql> GRANT SELECT ON sentinelx.asset               TO 'sx_analyst';
Query OK, 0 rows affected (0.01 sec)

mysql> GRANT SELECT ON sentinelx.vulnerability       TO 'sx_analyst';
Query OK, 0 rows affected (0.00 sec)

mysql> GRANT SELECT ON sentinelx.asset_vulnerability TO 'sx_analyst';
Query OK, 0 rows affected (0.00 sec)

mysql> GRANT SELECT ON sentinelx.attack_type         TO 'sx_analyst';
Query OK, 0 rows affected (0.00 sec)

mysql> GRANT SELECT ON sentinelx.mitigation          TO 'sx_analyst';
Query OK, 0 rows affected (0.00 sec)

mysql> GRANT SELECT ON sentinelx.threat_actor        TO 'sx_analyst';
Query OK, 0 rows affected (0.00 sec)

mysql> GRANT SELECT ON sentinelx.malware             TO 'sx_analyst';
Query OK, 0 rows affected (0.00 sec)

mysql> GRANT SELECT ON sentinelx.attack_event        TO 'sx_analyst';
Query OK, 0 rows affected (0.01 sec)

mysql> GRANT SELECT ON sentinelx.event_ioc           TO 'sx_analyst';
Query OK, 0 rows affected (0.00 sec)

mysql> GRANT SELECT, INSERT ON sentinelx.ioc         TO 'sx_analyst';
Query OK, 0 rows affected (0.01 sec)

mysql> GRANT SELECT, UPDATE ON sentinelx.v_my_incidents TO 'sx_analyst';
Query OK, 0 rows affected (0.00 sec)

mysql> GRANT SELECT ON sentinelx.v_attack_details    TO 'sx_analyst';
Query OK, 0 rows affected (0.00 sec)

mysql> GRANT EXECUTE ON PROCEDURE sentinelx.sp_assign_incident TO 'sx_analyst';
Query OK, 0 rows affected (0.00 sec)

mysql> GRANT SELECT ON sentinelx.v_asset_risk        TO 'sx_viewer';
Query OK, 0 rows affected (0.00 sec)

mysql> GRANT EXECUTE ON PROCEDURE sentinelx.sp_attack_report TO 'sx_viewer';
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE USER 'ananya.rao'@'localhost'  IDENTIFIED BY 'Sx#Admin2026'   DEFAULT ROLE 'sx_admin';
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE USER 'rahul.mehta'@'localhost' IDENTIFIED BY 'Sx#Analyst2026' DEFAULT ROLE 'sx_analyst';
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE USER 'divya.iyer'@'localhost'  IDENTIFIED BY 'Sx#Viewer2026'  DEFAULT ROLE 'sx_viewer';
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT user, host, default_role_user AS default_role
    -> FROM mysql.default_roles
    -> WHERE user IN ('ananya.rao', 'rahul.mehta', 'divya.iyer')
    -> ORDER BY user;
+-------------+-----------+--------------+
| user        | host      | default_role |
+-------------+-----------+--------------+
| ananya.rao  | localhost | sx_admin     |
| divya.iyer  | localhost | sx_viewer    |
| rahul.mehta | localhost | sx_analyst   |
+-------------+-----------+--------------+
3 rows in set (0.00 sec)
```

Fig. 100 Roles, grants and users

```
mysql> SHOW GRANTS FOR 'sx_viewer';
+-------------------------------------------------------------------------------+
| Grants for sx_viewer@%                                                        |
+-------------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `sx_viewer`@`%`                                         |
| GRANT SELECT ON `sentinelx`.`v_asset_risk` TO `sx_viewer`@`%`                 |
| GRANT EXECUTE ON PROCEDURE `sentinelx`.`sp_attack_report` TO `sx_viewer`@`%` |
+-------------------------------------------------------------------------------+
3 rows in set (0.00 sec)

mysql> SHOW GRANTS FOR 'sx_analyst';
+-----------------------------------------------------------------------------------------+
| Grants for sx_analyst@%                                                                 |
+-----------------------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `sx_analyst`@`%`                                                  |
| GRANT SELECT ON `sentinelx`.`asset_vulnerability` TO `sx_analyst`@`%`                   |
| GRANT SELECT ON `sentinelx`.`asset` TO `sx_analyst`@`%`                                 |
| GRANT SELECT ON `sentinelx`.`attack_event` TO `sx_analyst`@`%`                          |
| GRANT SELECT ON `sentinelx`.`attack_type` TO `sx_analyst`@`%`                           |
| GRANT SELECT ON `sentinelx`.`event_ioc` TO `sx_analyst`@`%`                             |
| GRANT SELECT, INSERT ON `sentinelx`.`ioc` TO `sx_analyst`@`%`                           |
| GRANT SELECT ON `sentinelx`.`malware` TO `sx_analyst`@`%`                               |
| GRANT SELECT ON `sentinelx`.`mitigation` TO `sx_analyst`@`%`                            |
| GRANT SELECT ON `sentinelx`.`threat_actor` TO `sx_analyst`@`%`                          |
| GRANT SELECT ON `sentinelx`.`v_attack_details` TO `sx_analyst`@`%`                      |
| GRANT SELECT, UPDATE ON `sentinelx`.`v_my_incidents` TO `sx_analyst`@`%`                |
| GRANT SELECT ON `sentinelx`.`vulnerability` TO `sx_analyst`@`%`                         |
| GRANT EXECUTE ON PROCEDURE `sentinelx`.`sp_assign_incident` TO `sx_analyst`@`%`         |
+-----------------------------------------------------------------------------------------+
14 rows in set (0.00 sec)

mysql> SHOW GRANTS FOR 'divya.iyer'@'localhost' USING 'sx_viewer';
+-----------------------------------------------------------------------------------------+
| Grants for divya.iyer@localhost                                                         |
+-----------------------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `divya.iyer`@`localhost`                                          |
| GRANT SELECT ON `sentinelx`.`v_asset_risk` TO `divya.iyer`@`localhost`                  |
| GRANT EXECUTE ON PROCEDURE `sentinelx`.`sp_attack_report` TO `divya.iyer`@`localhost` |
| GRANT `sx_viewer`@`%` TO `divya.iyer`@`localhost`                                       |
+-----------------------------------------------------------------------------------------+
4 rows in set (0.00 sec)
```

Fig. 101 SHOW GRANTS for the roles and for a user

```
mysql> GRANT SELECT ON sentinelx.network_log TO 'sx_viewer';
Query OK, 0 rows affected (0.03 sec)

mysql> SHOW GRANTS FOR 'sx_viewer';
+-----------------------------------------------------------------------------------+
| Grants for sx_viewer@%                                                            |
+-----------------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `sx_viewer`@`%`                                             |
| GRANT SELECT ON `sentinelx`.`network_log` TO `sx_viewer`@`%`                      |
| GRANT SELECT ON `sentinelx`.`v_asset_risk` TO `sx_viewer`@`%`                     |
| GRANT EXECUTE ON PROCEDURE `sentinelx`.`sp_attack_report` TO `sx_viewer`@`%`      |
+-----------------------------------------------------------------------------------+
4 rows in set (0.00 sec)

mysql> REVOKE SELECT ON sentinelx.network_log FROM 'sx_viewer';
Query OK, 0 rows affected (0.00 sec)

mysql> SHOW GRANTS FOR 'sx_viewer';
+-----------------------------------------------------------------------------------+
| Grants for sx_viewer@%                                                            |
+-----------------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `sx_viewer`@`%`                                             |
| GRANT SELECT ON `sentinelx`.`v_asset_risk` TO `sx_viewer`@`%`                     |
| GRANT EXECUTE ON PROCEDURE `sentinelx`.`sp_attack_report` TO `sx_viewer`@`%`      |
+-----------------------------------------------------------------------------------+
3 rows in set (0.00 sec)
```

Fig. 102 REVOKE: before and after

Logged in as the viewer, divya.iyer:

```
mysql> SELECT CURRENT_USER() AS logged_in_as, CURRENT_ROLE() AS active_role;
+----------------------+-----------------+
| logged_in_as         | active_role     |
+----------------------+-----------------+
| divya.iyer@localhost | `sx_viewer`@`%` |
+----------------------+-----------------+
1 row in set (0.00 sec)

mysql> SELECT hostname, criticality, attacks, not_blocked, open_vulns
    -> FROM v_asset_risk
    -> ORDER BY not_blocked DESC
    -> LIMIT 3;
+----------+-------------+---------+-------------+------------+
| hostname | criticality | attacks | not_blocked | open_vulns |
+----------+-------------+---------+-------------+------------+
| web-01   |           5 |       7 |           4 |          1 |
| db-01    |           5 |       7 |           4 |          0 |
| web-02   |           4 |       5 |           2 |          1 |
+----------+-------------+---------+-------------+------------+
3 rows in set (0.00 sec)

mysql> CALL sp_attack_report('2026-10-20', '2026-10-28');
+--------------------------+---------+---------+-------+
| type_name                | attacks | blocked | worst |
+--------------------------+---------+---------+-------+
| Exploit Public-Facing App |       4 |       1 |     9 |
| Brute Force              |       1 |       1 |     4 |
| DDoS                     |       1 |       1 |     7 |
| Phishing                 |       1 |       1 |     5 |
| Port Scan                |       1 |       1 |     3 |
| Ransomware               |       1 |       0 |    10 |
+--------------------------+---------+---------+-------+
6 rows in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

mysql> SELECT * FROM attack_event LIMIT 1;
ERROR 1142 (42000): SELECT command denied to user 'divya.iyer'@'localhost' for table 'attack_event'

mysql> DELETE FROM attack_event WHERE event_id = 1;
ERROR 1142 (42000): DELETE command denied to user 'divya.iyer'@'localhost' for table 'attack_event'

mysql> UPDATE v_asset_risk SET criticality = 1 WHERE asset_id = 1;
ERROR 1142 (42000): UPDATE command denied to user 'divya.iyer'@'localhost' for table 'v_asset_risk'
```

Fig. 103 Least privilege as divya.iyer (viewer)

Logged in as the analyst, rahul.mehta. He sees only his 3 incidents, can update their status through the view, cannot reassign one to another analyst (CHECK OPTION), and cannot read the incident table or drop tables:

```
mysql> SELECT CURRENT_USER() AS logged_in_as, CURRENT_ROLE() AS active_role;
+-----------------------+-----------------+
| logged_in_as          | active_role     |
+-----------------------+-----------------+
| rahul.mehta@localhost | `sx_analyst`@`%` |
+-----------------------+-----------------+
1 row in set (0.00 sec)

mysql> SELECT incident_id, title, priority, status FROM v_my_incidents;
+-------------+----------------------------------------+----------+---------------+
| incident_id | title                                  | priority | status        |
+-------------+----------------------------------------+----------+---------------+
|           1 | Brute force on web-01 from 198.51.100.23 | HIGH     | CLOSED        |
|           3 | Iron Kestrel intrusion on db-01        | CRITICAL | CONTAINED     |
|           7 | Brute force on web-01 from 198.51.100.45 | HIGH     | INVESTIGATING |
+-------------+----------------------------------------+----------+---------------+
3 rows in set (0.00 sec)

mysql> UPDATE v_my_incidents SET status = 'CONTAINED' WHERE incident_id = 7;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT incident_id, status FROM v_my_incidents WHERE incident_id = 7;
+-------------+-----------+
| incident_id | status    |
+-------------+-----------+
|           7 | CONTAINED |
+-------------+-----------+
1 row in set (0.00 sec)

mysql> UPDATE v_my_incidents SET assigned_to = 3 WHERE incident_id = 7;
ERROR 1369 (HY000): CHECK OPTION failed 'sentinelx.v_my_incidents'

mysql> SELECT * FROM incident LIMIT 1;
ERROR 1142 (42000): SELECT command denied to user 'rahul.mehta'@'localhost' for table 'incident'

mysql> DROP TABLE asset;
ERROR 1142 (42000): DROP command denied to user 'rahul.mehta'@'localhost' for table 'asset'
```

Fig. 104 Least privilege as rahul.mehta (analyst)

```
mysql> SELECT CURRENT_USER() AS logged_in_as, CURRENT_ROLE() AS active_role;
+----------------------+----------------+
| logged_in_as         | active_role    |
+----------------------+----------------+
| ananya.rao@localhost | `sx_admin`@`%` |
+----------------------+----------------+
1 row in set (0.00 sec)

mysql> SELECT COUNT(*) AS incidents FROM incident;
+-----------+
| incidents |
+-----------+
|         9 |
+-----------+
1 row in set (0.00 sec)
```

Fig. 105 Admin login as ananya.rao

13. Transactions

SAVEPOINT and COMMIT. A new indicator is inserted and a savepoint is set. A wrong priority change follows, is rolled back to the savepoint, and the transaction is committed. The indicator is kept and the priority change is not.

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO ioc (ioc_type, ioc_value, actor_id, valid_from, recorded_from)
    -> VALUES ('domain', 'kestrel-drop.example', 4, '2026-10-20 00:00:00', '2026-10-27 12:00:00');
Query OK, 1 row affected (0.00 sec)

mysql> SAVEPOINT after_ioc;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE incident SET priority = 'LOW' WHERE incident_id = 3;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT incident_id, priority FROM incident WHERE incident_id = 3;
+-------------+----------+
| incident_id | priority |
+-------------+----------+
|           3 | LOW      |
+-------------+----------+
1 row in set (0.00 sec)

mysql> ROLLBACK TO SAVEPOINT after_ioc;
Query OK, 0 rows affected (0.00 sec)

mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT ioc_version_id, ioc_type, ioc_value FROM ioc WHERE ioc_value = 'kestrel-drop.example';
+----------------+----------+----------------------+
| ioc_version_id | ioc_type | ioc_value            |
+----------------+----------+----------------------+
|             14 | domain   | kestrel-drop.example |
+----------------+----------+----------------------+
1 row in set (0.00 sec)

mysql> SELECT incident_id, priority FROM incident WHERE incident_id = 3;
+-------------+----------+
| incident_id | priority |
+-------------+----------+
|           3 | CRITICAL |
+-------------+----------+
1 row in set (0.00 sec)
```

Fig. 106 START TRANSACTION, SAVEPOINT, ROLLBACK TO SAVEPOINT and COMMIT

ROLLBACK. This is how log batches are loaded: the second line of the batch points to an asset that does not exist, so the whole batch is rolled back and the log count is unchanged (50,021 before and after).

```
mysql> SELECT COUNT(*) AS logs_before FROM network_log;
+-------------+
| logs_before |
+-------------+
|       50021 |
+-------------+
1 row in set (0.02 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO network_log (log_time, src_ip, dst_asset_id, dst_port, protocol, action)
    -> VALUES ('2026-10-28 09:00:00', '203.0.113.5', 1, 443, 'TCP', 'allowed');
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO network_log (log_time, src_ip, dst_asset_id, dst_port, protocol, action)
    -> VALUES ('2026-10-28 09:00:01', '203.0.113.5', 99, 443, 'TCP', 'allowed');
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails
  (`sentinelx`.`network_log`, CONSTRAINT `network_log_ibfk_1` FOREIGN KEY (`dst_asset_id`) REFERENCES
  `asset` (`asset_id`))

mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT COUNT(*) AS logs_after FROM network_log;
+------------+
| logs_after |
+------------+
|      50021 |
+------------+
1 row in set (0.01 sec)
```

Fig. 107 Failed batch rolled back as a whole

14. Indexes and EXPLAIN

Besides the primary keys and the indexes MySQL creates for foreign keys, we added three: idx_src_time (src_ip, log_time) on network_log for retro-hunts, and idx_time and idx_src on attack_event for the dashboard and search.

```
mysql> SELECT table_name, index_name,
    ->        GROUP_CONCAT(column_name ORDER BY seq_in_index) AS columns_in_index,
    ->        IF(non_unique = 0, 'yes', 'no') AS is_unique
    -> FROM information_schema.statistics
    -> WHERE table_schema = 'sentinelx'
    ->   AND table_name IN ('network_log', 'attack_event')
    -> GROUP BY table_name, index_name, non_unique
    -> ORDER BY table_name, index_name;
+--------------+--------------+------------------+-----------+
| TABLE_NAME   | INDEX_NAME   | columns_in_index | is_unique |
+--------------+--------------+------------------+-----------+
| attack_event | actor_id     | actor_id         | no        |
| attack_event | asset_id     | asset_id         | no        |
| attack_event | cve_id       | cve_id           | no        |
| attack_event | idx_src      | src_ip           | no        |
| attack_event | idx_time     | event_time       | no        |
| attack_event | malware_id   | malware_id       | no        |
| attack_event | PRIMARY      | event_id         | yes       |
| attack_event | type_id      | type_id          | no        |
| network_log  | dst_asset_id | dst_asset_id     | no        |
| network_log  | idx_src_time | src_ip,log_time  | no        |
| network_log  | PRIMARY      | log_id           | yes       |
+--------------+--------------+------------------+-----------+
11 rows in set (0.08 sec)
```

Fig. 108 Indexes on network_log and attack_event

To measure idx_src_time we dropped it, ran EXPLAIN on the retro-hunt lookup, then created it again and repeated the EXPLAIN.

```
mysql> ALTER TABLE network_log DROP INDEX idx_src_time;
Query OK, 0 rows affected (0.06 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> ANALYZE TABLE network_log;
+----------------------+---------+----------+----------+
| Table                | Op      | Msg_type | Msg_text |
+----------------------+---------+----------+----------+
| sentinelx.network_log | analyze | status   | OK       |
+----------------------+---------+----------+----------+
1 row in set (0.01 sec)

mysql> EXPLAIN SELECT log_id, log_time, dst_asset_id
    -> FROM network_log
    -> WHERE src_ip = '198.51.100.77'
    ->   AND log_time >= '2026-09-15' AND log_time < '2026-10-28'\G
*************************** 1. row ***************************
           id: 1
  select_type: SIMPLE
        table: network_log
   partitions: NULL
         type: ALL
possible_keys: NULL
          key: NULL
      key_len: NULL
          ref: NULL
         rows: 50163
     filtered: 1.11
        Extra: Using where
1 row in set, 1 warning (0.00 sec)

mysql> SELECT COUNT(*) AS hits FROM network_log
    -> WHERE src_ip = '198.51.100.77'
    ->   AND log_time >= '2026-09-15' AND log_time < '2026-10-28';
+------+
| hits |
+------+
|    8 |
+------+
1 row in set (0.01 sec)
```

Fig. 109 Without the index: full table scan

```
mysql> CREATE INDEX idx_src_time ON network_log (src_ip, log_time);
Query OK, 0 rows affected (0.54 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> ANALYZE TABLE network_log;
+----------------------+---------+----------+----------+
| Table                | Op      | Msg_type | Msg_text |
+----------------------+---------+----------+----------+
| sentinelx.network_log | analyze | status   | OK       |
+----------------------+---------+----------+----------+
1 row in set (0.02 sec)

mysql> EXPLAIN SELECT log_id, log_time, dst_asset_id
    -> FROM network_log
    -> WHERE src_ip = '198.51.100.77'
    ->   AND log_time >= '2026-09-15' AND log_time < '2026-10-28'\G
*************************** 1. row ***************************
           id: 1
  select_type: SIMPLE
        table: network_log
   partitions: NULL
         type: range
possible_keys: idx_src_time
          key: idx_src_time
      key_len: 187
          ref: NULL
         rows: 8
     filtered: 100.00
        Extra: Using index condition
1 row in set, 1 warning (0.00 sec)

mysql> SELECT COUNT(*) AS hits FROM network_log
    -> WHERE src_ip = '198.51.100.77'
    ->   AND log_time >= '2026-09-15' AND log_time < '2026-10-28';
+------+
| hits |
+------+
|    8 |
+------+
1 row in set (0.00 sec)
```

Fig. 110 With the index: range scan

	Without idx_src_time	**With idx_src_time**
Access type	ALL (full table scan)	range
Index used	none	idx_src_time
Rows examined (estimate)	50,163	8
Rows actually matching	8	8

The index lets MySQL jump straight to the 8 lines from that IP instead of reading all 50,021 rows. With 50,000 rows both runs still finish in about 0.01 seconds, but a full scan grows with the size of the table while the range scan only reads the matching rows, so the gap widens as logs accumulate.

15. Design Changes since Review 1

The professor's feedback on Review 1 was that the ER diagram was cluttered. These are all the changes made since then, with the reason for each.

#	Change	Reason
1	ER diagram redrawn with no crossing lines, grouped by area, strict Chen notation	Review 1 feedback: the diagram was hard to read
2	23 attributes removed (for example asset.os, department, country_code; vulnerability.vendor, product, description; ioc.confidence, source)	No query, trigger or page used them
3	vulnerability.severity and malware.category removed	Transitive dependencies (cvss -> severity, family -> category) broke 3NF
4	retro_hunt_run.logs_matched and assets_touched removed; view v_retro_hunt_summary added	They were counts of rows in retro_hit and could go out of sync
5	PostgreSQL 16 replaced by MySQL 8.0	Course lab uses MySQL; all DDL rewritten
6	Monthly partitioning of logs and attacks replaced by indexes	MySQL does not allow foreign keys on partitioned tables; section 14 shows the index is enough
7	tstzrange periods on ioc became valid_from/valid_to and recorded_from/recorded_to	MySQL has no range types
8	Exclusion constraint on ioc replaced by trigger trg_ioc_no_overlap	MySQL has no exclusion constraints
9	INET columns became VARCHAR(45); fixed lists became ENUM	MySQL has no INET type; ENUM uses 1 byte and rejects other values
10	Link tables reference attacks and logs by event_id and log_id alone	Their keys no longer include the partition time column
11	Added v_attack_details, v_asset_risk, v_my_incidents; 2 functions; 4 procedures (2 with cursors); 4 triggers; 3 roles and users	Review 2 requirements, and least-privilege access for analysts and viewers

Normalisation steps

Step	What we did
Starting point	One flat row per attack with time, source IP, host details, attack name, attacker, malware, CVE and score, and a list of IOCs like "198.51.100.23, evil-1.example".
1NF	Split the IOC list so each row holds one indicator.
2NF	Attack details depend only on the event and indicator details only on the indicator, so they went into ATTACK_EVENT, IOC and the link table EVENT_IOC.
3NF	Host, attacker, malware, CVE, attack type and analyst details each moved to their own table; the two transitive dependencies in change 3 were removed.

16. Review 2 Requirements Checklist

#	Mandatory component	Evidence	Where	Status
1	CREATE TABLE scripts + all constraints	01_ddl.sql runs cleanly on MySQL 8.0.46: 16 tables, no errors	Section 9	Done

#	Mandatory component	Evidence	Where	Status
2	INSERT, UPDATE, DELETE	02_dml.sql, row-count table (50,021 logs, 30 attacks)	Section 10	Done
3	Basic, aggregate, join, subquery, set-operation queries	40 queries (A1-G3), 20+ required	Section 11	Done
4	Views	5 views (3 new + 2 from Review 1)	12.2	Done
5	Transactions (COMMIT/ROLLBACK/SAVEPOINT)	Savepoint demo and failed-batch rollback	Section 13	Done
6	Procedures / functions	4 procedures, 2 functions, all called successfully	12.3, 12.4	Done
7	Triggers	4 triggers, each shown firing	12.5	Done
8	Indexes + one EXPLAIN comparison	ALL / 50,163 rows vs range / 8 rows	Section 14	Done
9	Execution screenshots	Every screenshot is the real mysql client output, with its SQL	Throughout	Done
10	Design changes since Review 1	Change log with reasons	Section 15	Done
+	Complex business queries (10+)	11 queries with expected vs actual output	12.1	Done
+	Cursors (2+) and SQL security	2 cursors; 3 roles, 3 users, GRANT/REVOKE, least-privilege errors	12.6, 12.7	Done

17. Limitations and Future Scope

- The data is a test set, not real traffic. Attacker IPs come from documentation-only address ranges and the attacker and malware names are made up; the CVE IDs are real.
- Retro-hunts only work for IP addresses. Domains and file hashes would need DNS and file logs.
- MySQL has no exclusion constraint, so the rule that indicator versions never overlap lives in a trigger.
- Very high log volumes would need a streaming pipeline (for example Kafka) and archive tables for old logs.

18. Conclusion

Review 1 gave us a clean 3NF design; Review 2 shows it working. Every statement in this report ran on MySQL 8.0.46 and the screenshots are its real output. Doing that taught us things the design alone did not: the NOT IN query that silently returns nothing because of one NULL, the index that turns a 50,163-row scan into an 8-row range read, and the CHECK OPTION that stops an analyst from handing an incident to someone else. Our two own ideas, retro-hunting and the time machine, turned out to be one trigger and two pairs of date columns, which is a good reminder that careful table design does most of the work.

19. Work Break Down

Team Member Registration Number	Name	Work Assigned
24BCI0061	Shlok Gupta	Retro-hunt and IOC triggers, indexes and EXPLAIN, transactions, FastAPI backend

Team Member Registration Number	Name	Work Assigned
24BCI0086	Shreya Bhushan	ER diagram, normalisation, DDL and constraints, views, stored procedures and cursors, SQL security
24BDS0158	Yashneil	Test data and DML, SQL queries A-G, complex business queries, functions, React dashboard, this report

20. Review Evaluation

COMPONENT	MARKS	SHLOK GUPTA	SHREYA BHUSHAN	YASHNEIL
TOTAL	**10 MARKS**			